

Organizzazione a pipeline

Alessandro Pellegrini
a.pellegrini@ing.uniroma2.it

Motivazioni per una nuova architettura

- L'organizzazione a multiciclo è efficace perché permette il riuso dell'hardware per svolgere compiti differenti
- Tuttavia è un'organizzazione completamente *sequenziale*
 - In un istante di tempo, c'è una sola istruzione (meglio: microoperazione) in esecuzione nella CPU (*in-flight operation*)
- Il tempo necessario all'esecuzione di un programma è quindi:

$$T_{exec} = n_{cicli} \cdot T_{clock} \text{ equivalente a } T_{exec} = \frac{n_{cicli}}{f_{clock}}$$

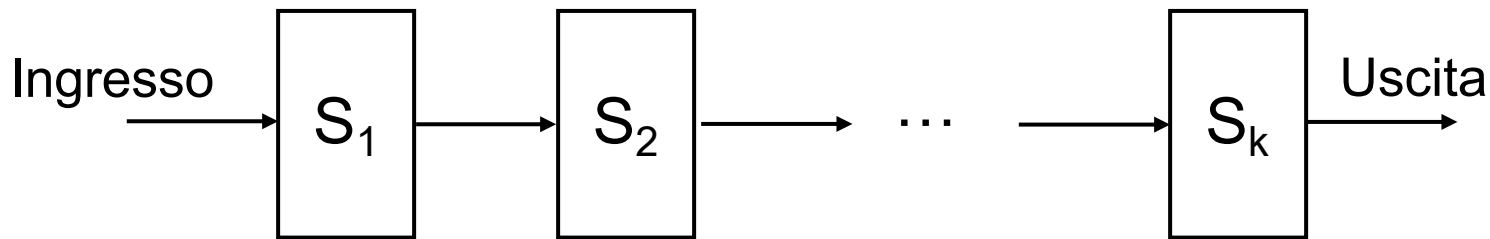
- Un'approssimazione di n_{cicli} può essere data da:

$$n_{cicli} = n_{insn} \cdot CPI$$

- Come possiamo ridurre T_{exec} ?

Il pipelining

- È una tecnica per progettare un'architettura hardware che:
 - consenta il miglioramento delle prestazioni del processore
 - si basa sulla *sovrapposizione* dell'esecuzione di *più istruzioni* appartenenti ad un flusso di esecuzione sequenziale
- Il lavoro svolto dal processore è suddiviso in *passi (stadi della pipeline)* che richiedono una frazione del tempo d'esecuzione di un'intera istruzione
- Gli stadi sono connessi sequenzialmente per formare la pipeline
- Le istruzioni vengono elaborate nei vari stadi secondo l'ordine previsto

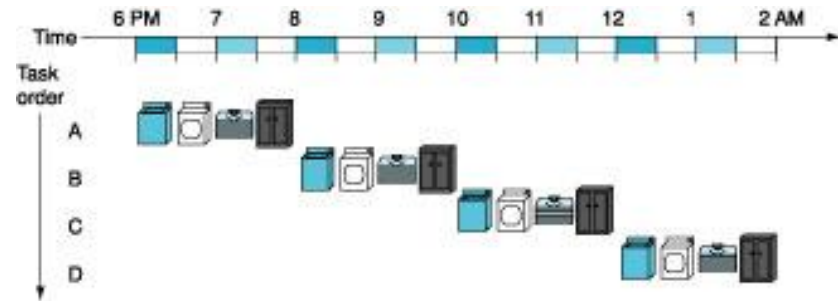


Un esempio

- Attività elementari per fare il bucato:

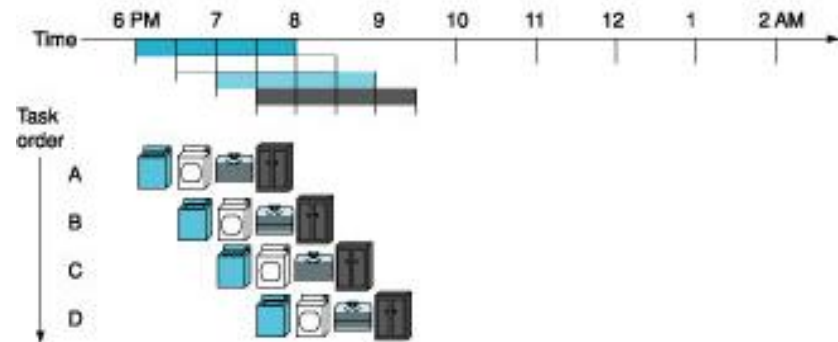
- Lavaggio 🧺
- Asciugatura 🧺
- Stiratura 🧺
- Riordino 🧺

Esecuzione sequenziale



- Nella soluzione con pipeline viene avviato il ciclo di lavaggio successivo mentre quello precedente è ancora in esecuzione in un'altra fase

Soluzione con pipeline



Set istruzioni del processore z64 a pipeline

- Per semplicità, implementeremo in hardware un sottoinsieme delle istruzioni del processore z64

Class	Instruction	Syntax	Semantics
HW	Non-operational	nop	No operation (beyond the fetch phase)
L/A	Sum	addq %regsorg, %regdest	regdest = regsorg + regdest
	Subtraction	subq %regsorg, %regdest	regdest = regdest - regsorg
	Logic product	andq %regsorg, %regdest	regdest = regsorg and regdest
	Logic sum	orq %regsorg, %regdest	regdest = regsorg or regdest
	Logic negation	notq %register	register = not register
L/S	Loading of word	movq offset(%regbase), %regdest	regdest = memory[offset+regbase]
	Storage of word	movq %regsorg, offset(%regbase)	memory[offset+regbase] = regsorg
J	jump if flag X == 1	jX displacement	if flag X == 1 then RIP = RIP + displacement

- Il formato istruzione è identico a quello del processore multiciclo (ma supportiamo solo istruzioni a 64 bit)

Passi d'esecuzione per le istruzioni

- Le istruzioni considerate richiedono un numero di passi differente:

	nop	L/A	Load	Store	Cond. Jump
<i>fetch</i>	prelievo	prelievo	prelievo	prelievo	prelievo
<i>decode</i>	decodifica	decodifica	decodifica	decodifica e accesso a banco dei registri	decodifica
<i>execute</i>		esecuzione	calcolo indirizzo effettivo	calcolo indirizzo effettivo	calcolo indirizzo destinazione
<i>memory</i>			accesso a memoria	accesso a memoria	
<i>write back</i>		scrittura risultato	scrittura risultato		scrittura risultato
	2	4	5	4	4

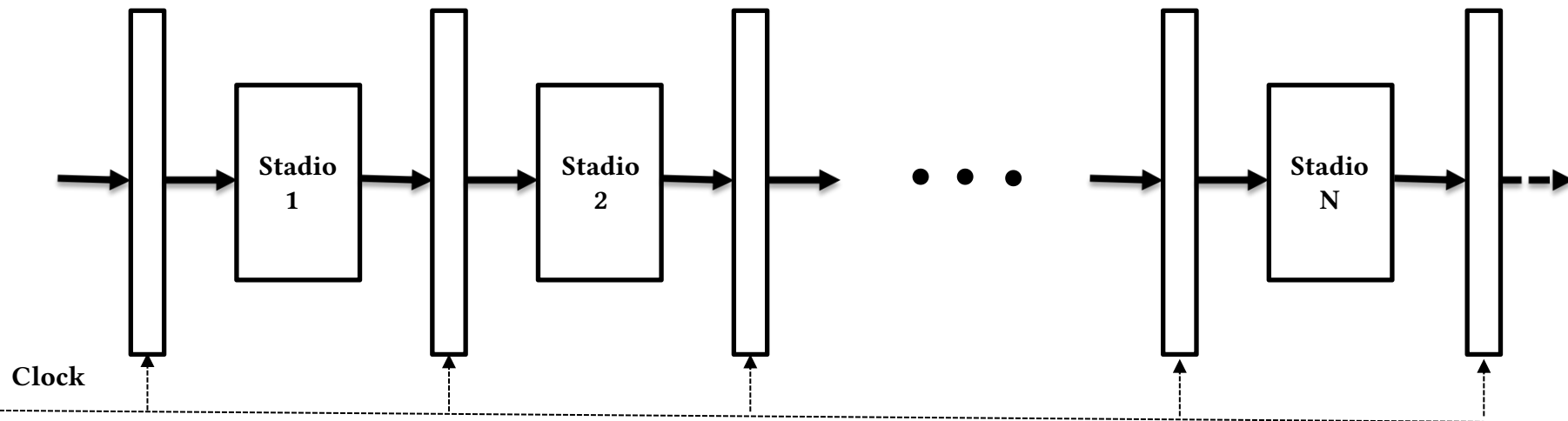
- L'organizzazione a multiciclo sfruttava questa caratteristica per migliorare le prestazioni
- In un'architettura a pipeline occorre che tutte le istruzioni abbiano lo stesso numero di passi d'esecuzione

Vantaggi dell'organizzazione a pipeline

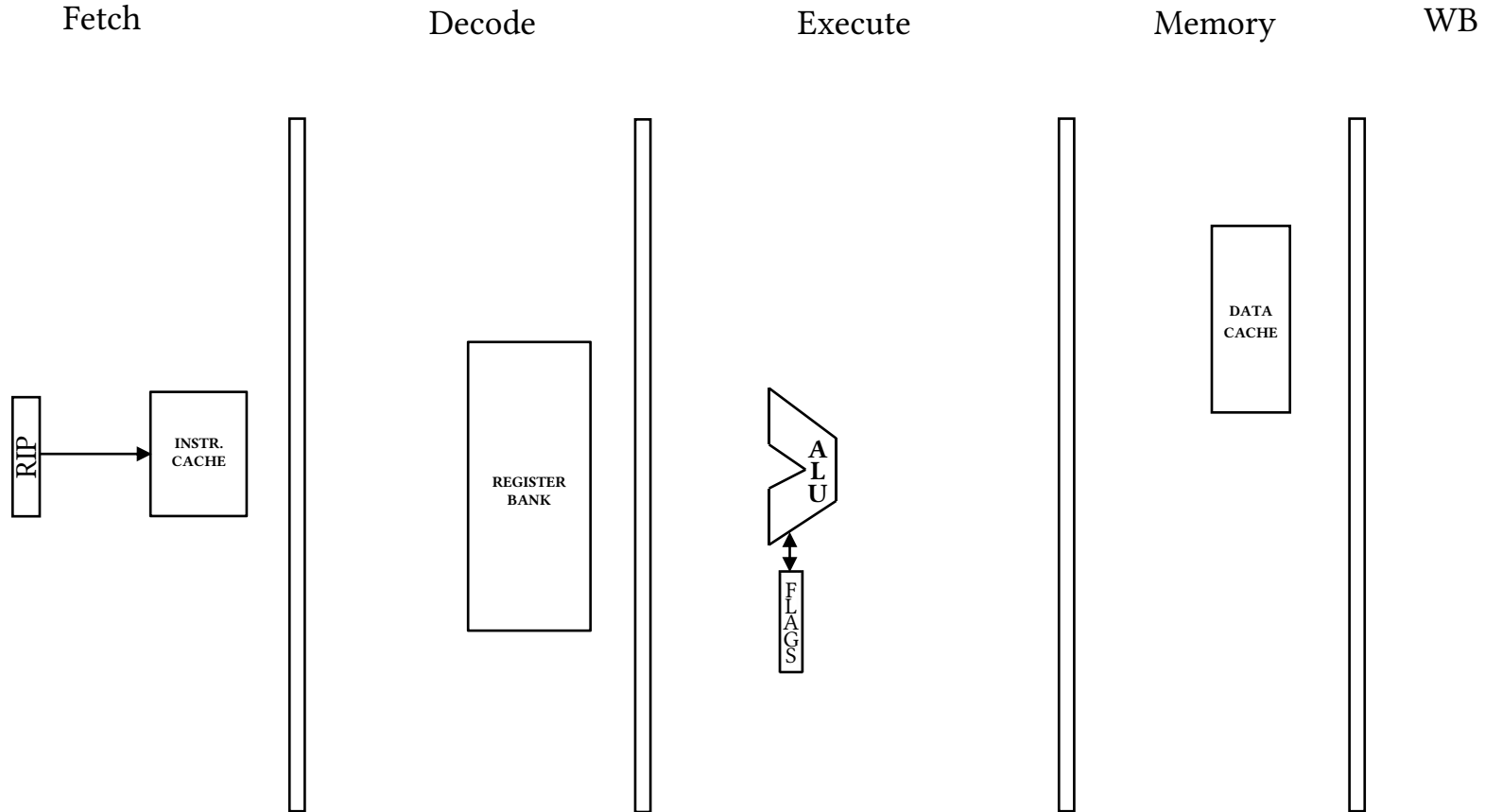
- Il critical path di un singolo stadio della pipeline più breve (f_{clock} aumenta)
- In ogni ciclo di clock, ci sono fino a 5 istruzioni in esecuzione parallela (dopo un transiente iniziale, $CPI \approx 1$, almeno in teoria)
- L'esecuzione di un singolo programma viene potenzialmente ridotta
- Questi vantaggi tendono ad aumentare all'aumentare del numero di stadi nella pipeline (*profondità* della pipeline)

Schema di principio delle architetture pipeline

- Gli stadi devono essere macchine indipendenti
- È necessario disaccoppiarle con dei registri tampone (*registri di pipeline*)
- Istruzioni diverse possono richiedere lo stesso hardware (*replicazione*)

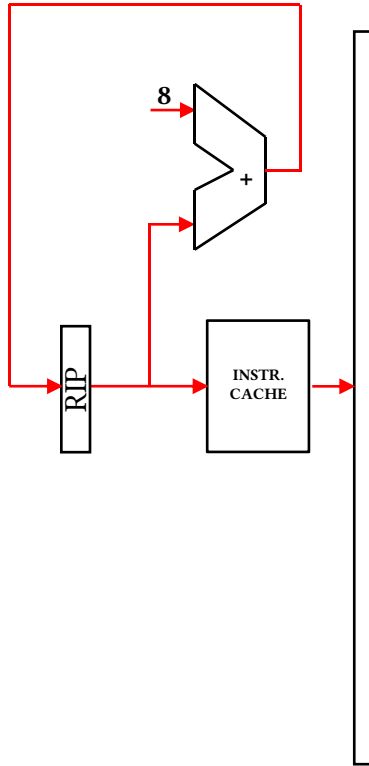


Progettazione del datapath: elementi di base

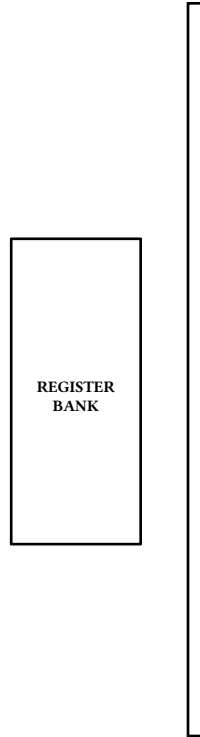


Istruzioni L/A: fase di fetch

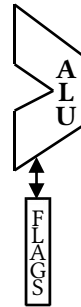
Fetch



Decode



Execute



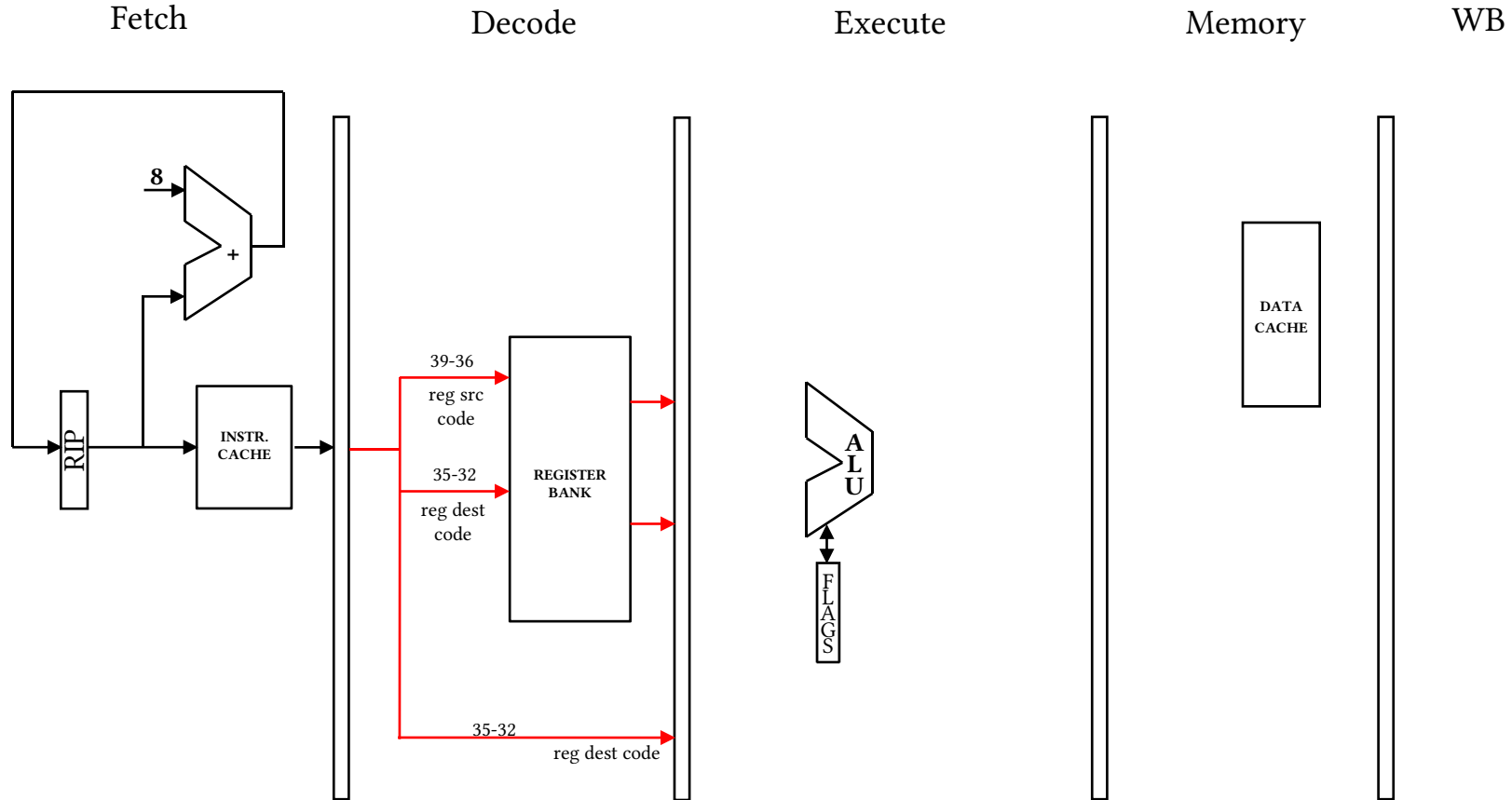
Memory



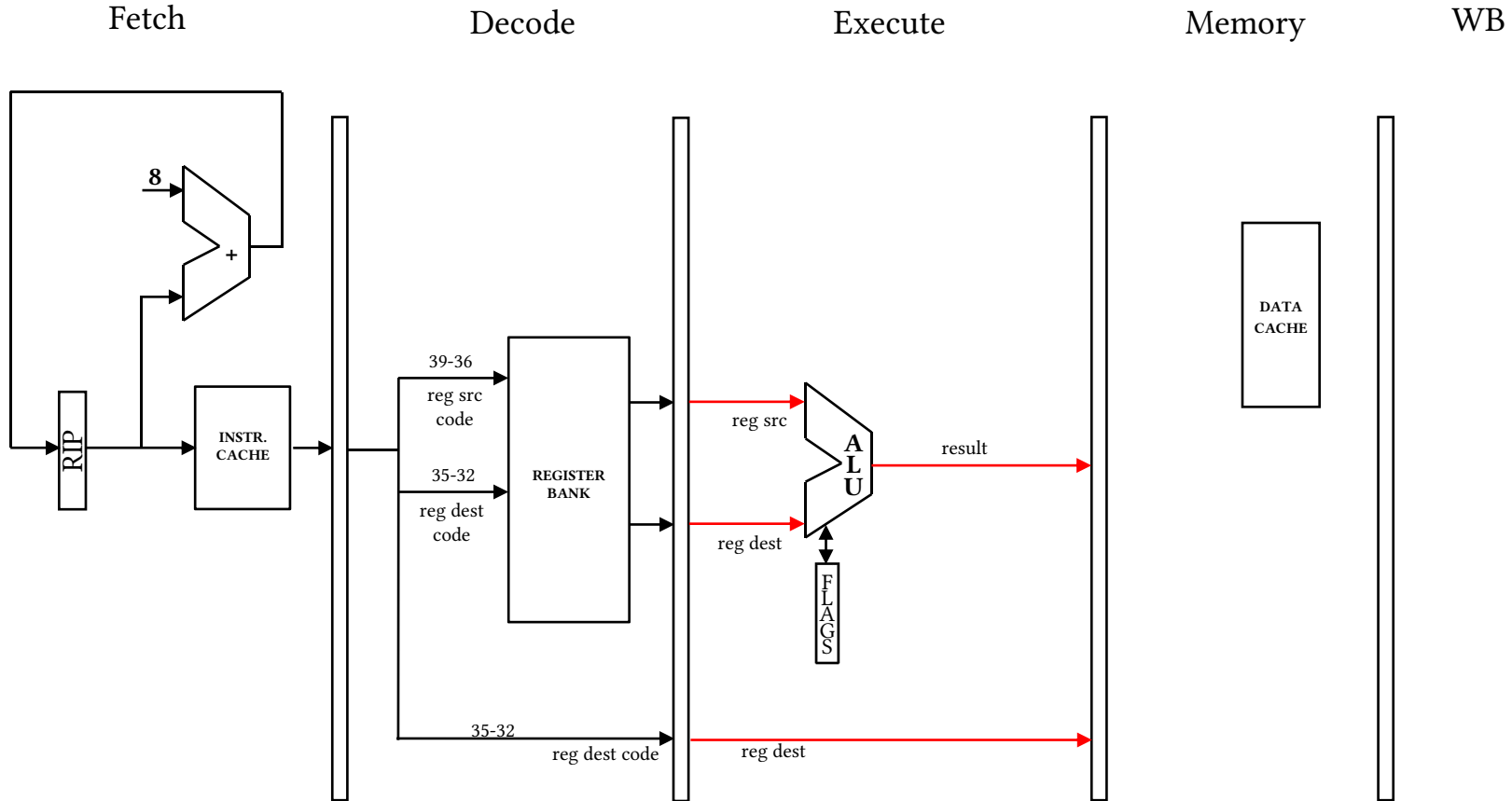
WB



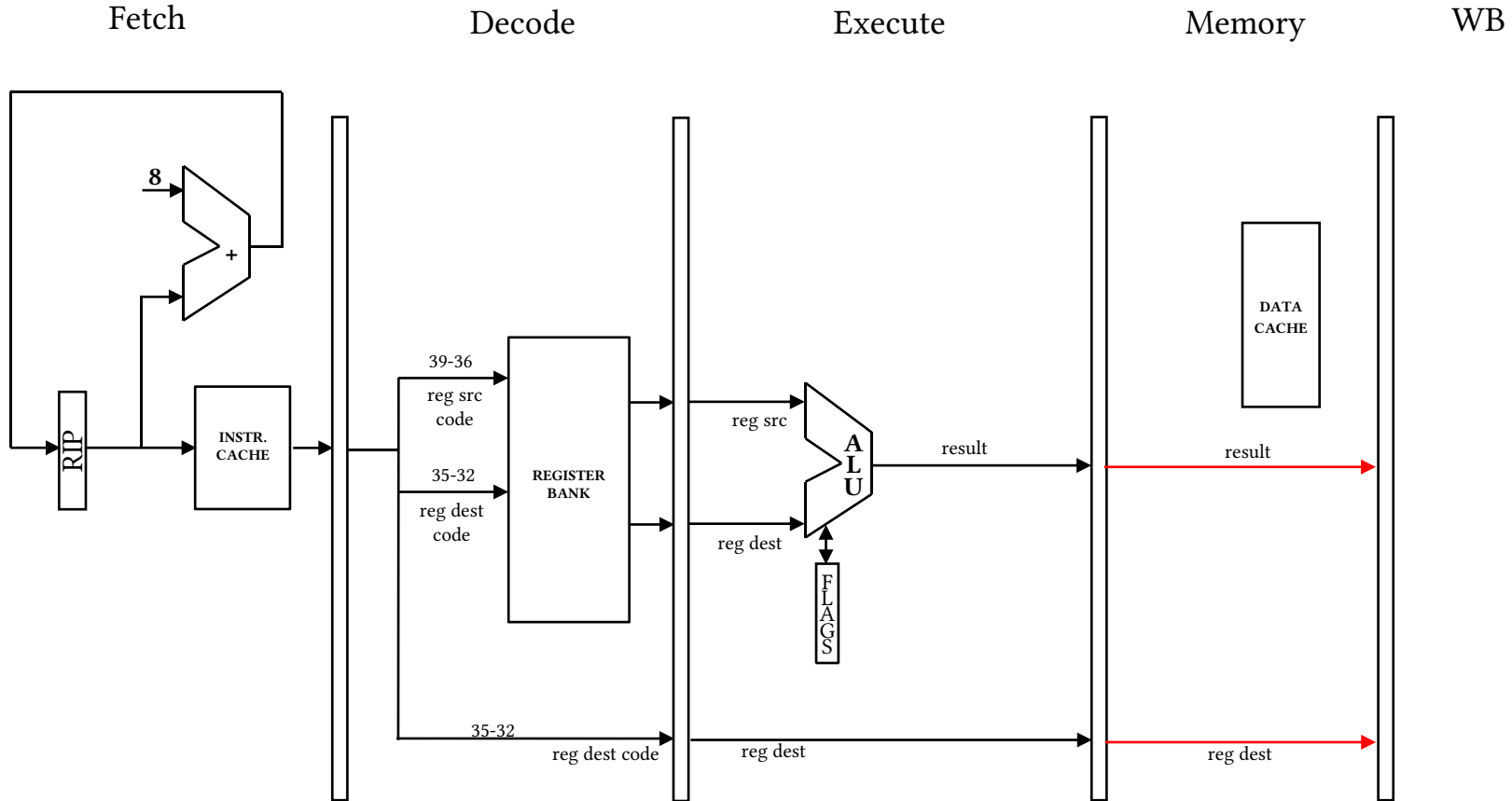
Istruzioni L/A: fase di decode



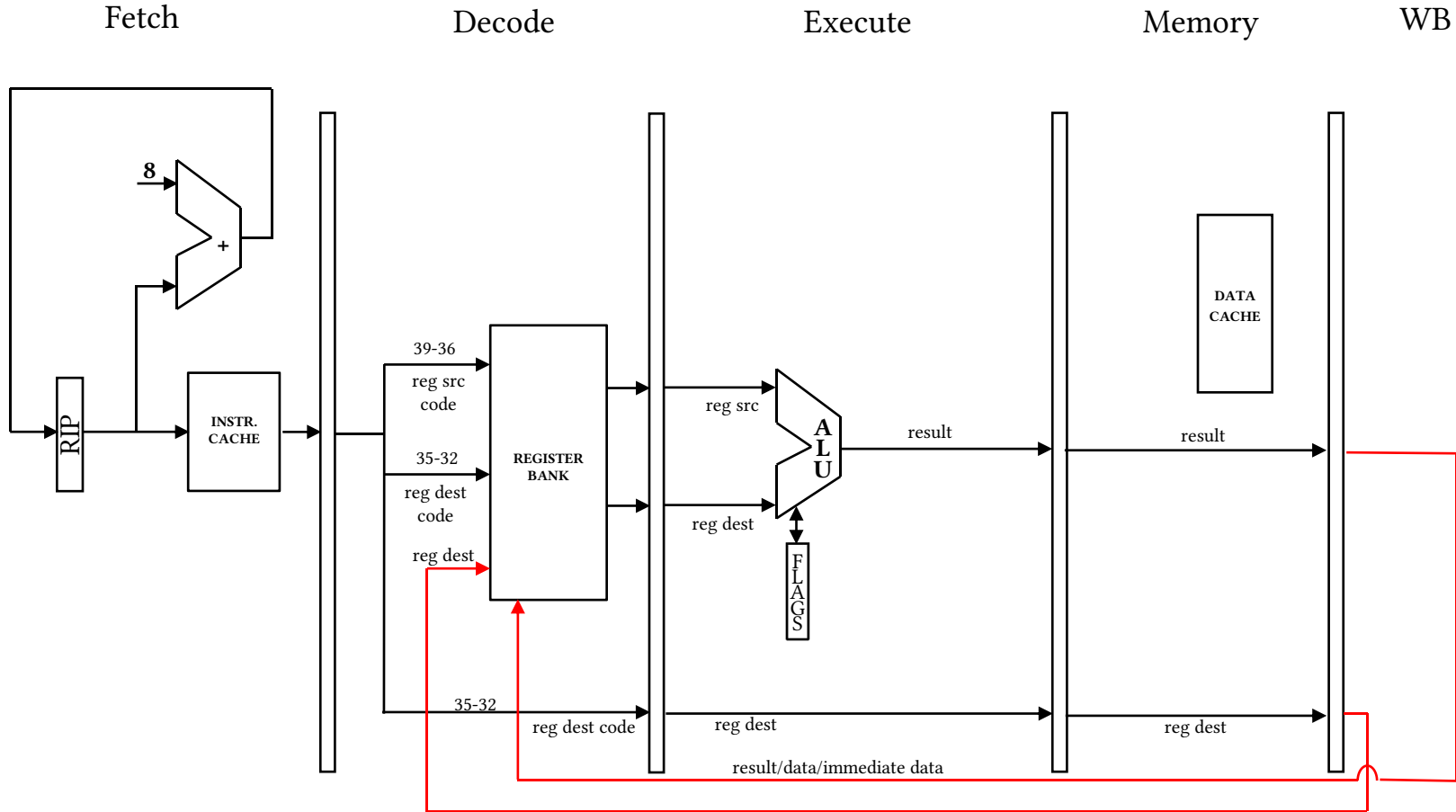
Istruzioni L/A: fase di execute



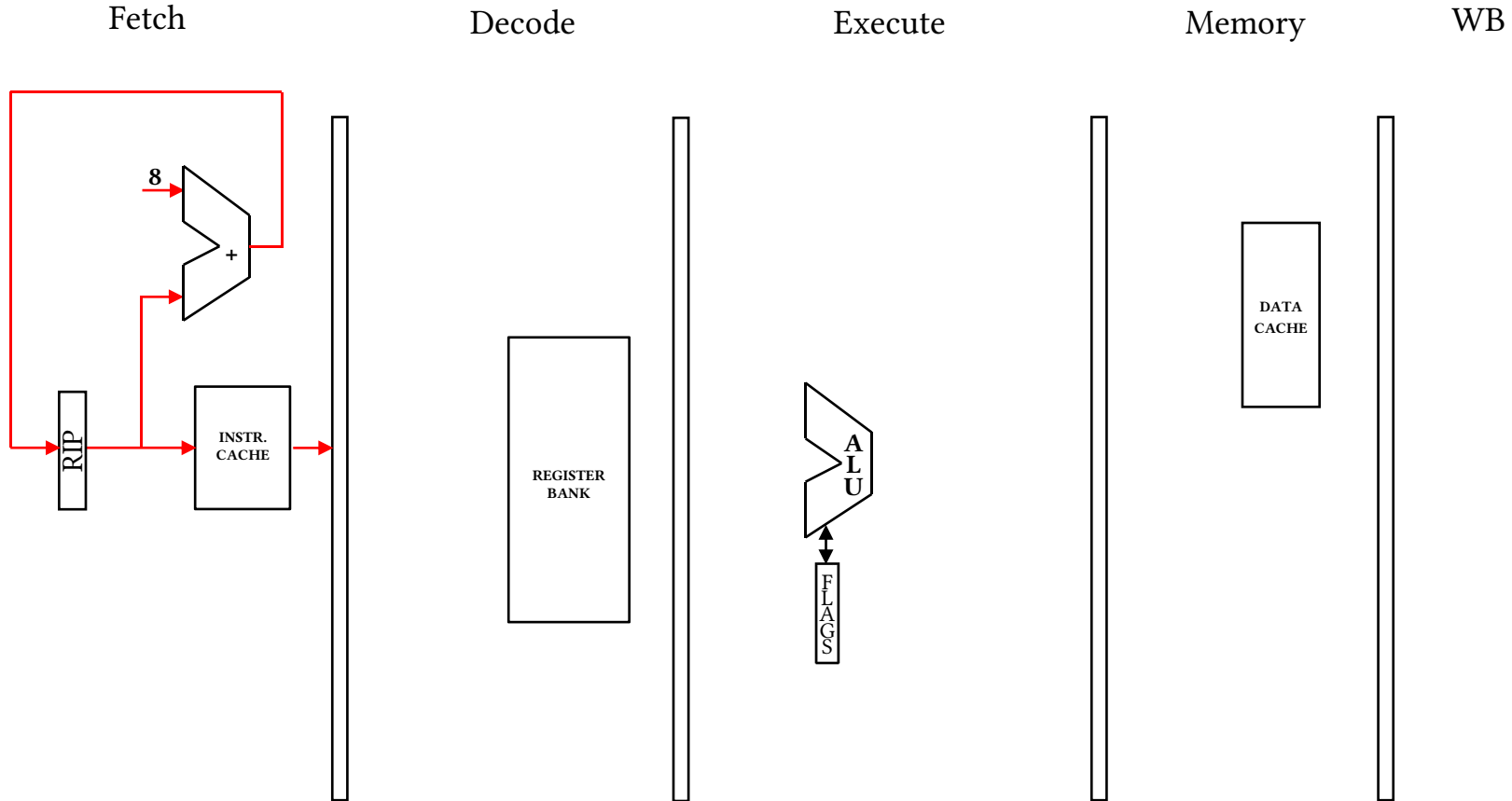
Istruzioni L/A: fase di memory



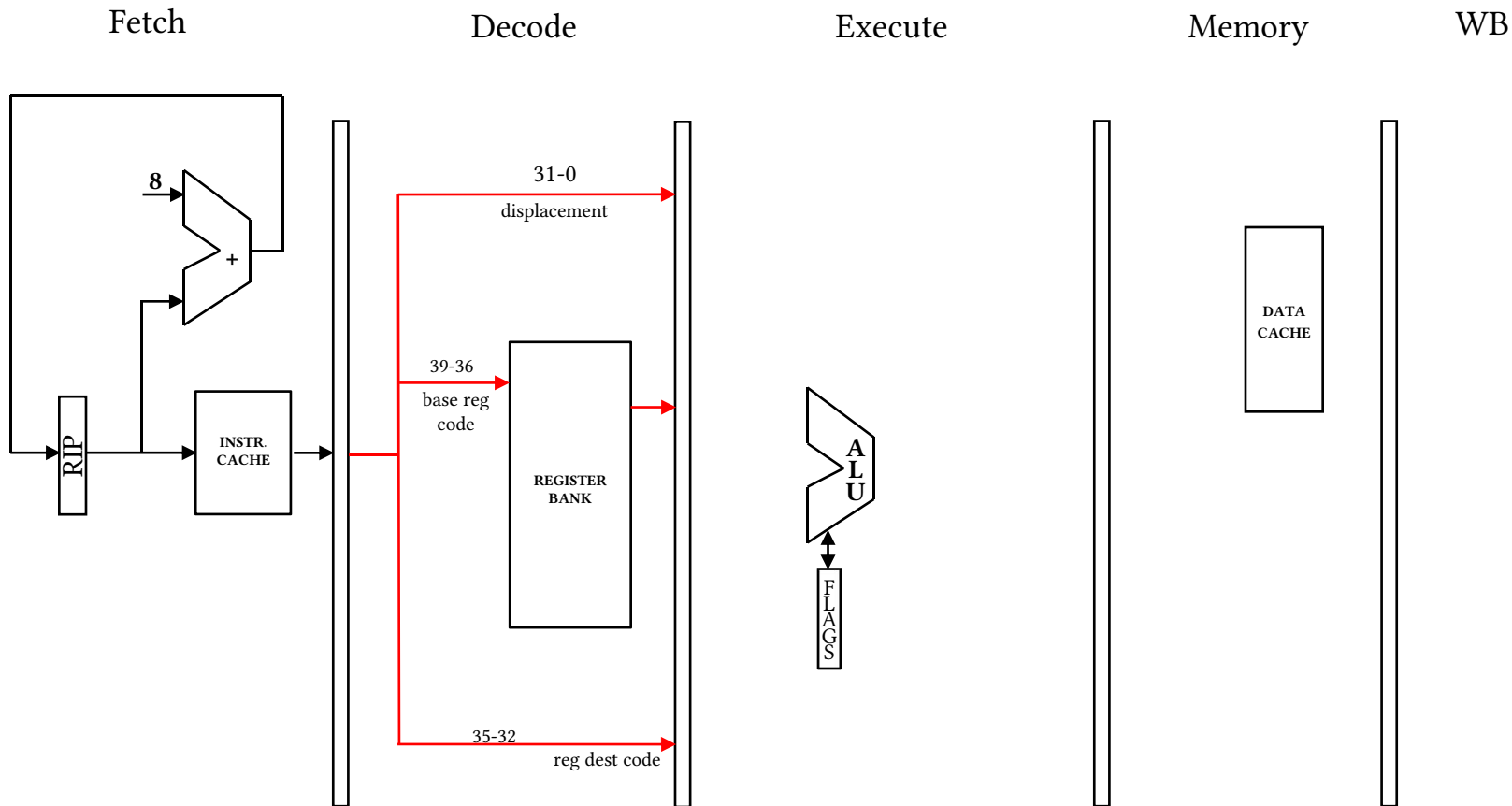
Istruzioni L/A: fase di write back



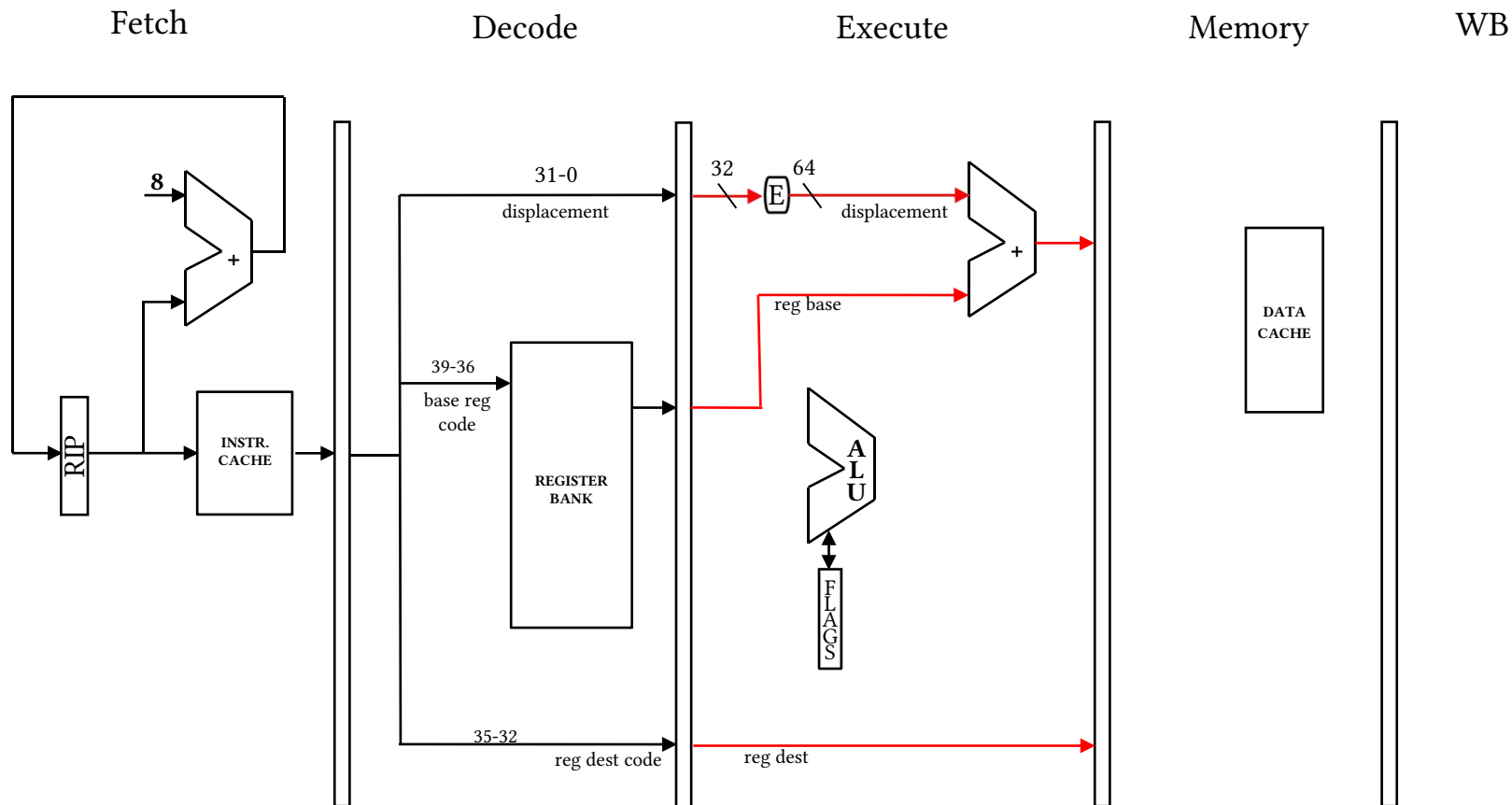
Istruzione Load: fase di fetch



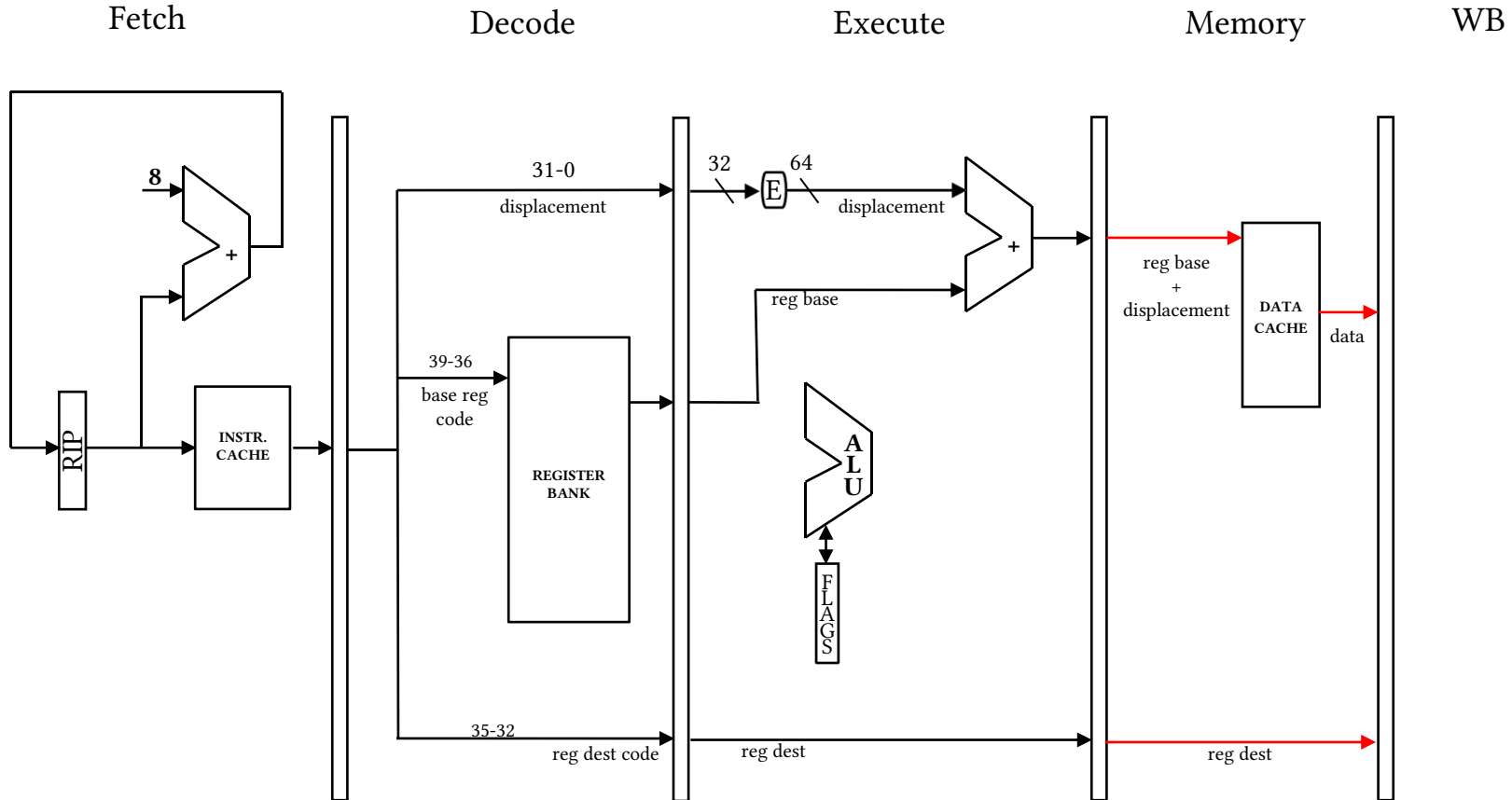
Istruzione Load: fase di decode



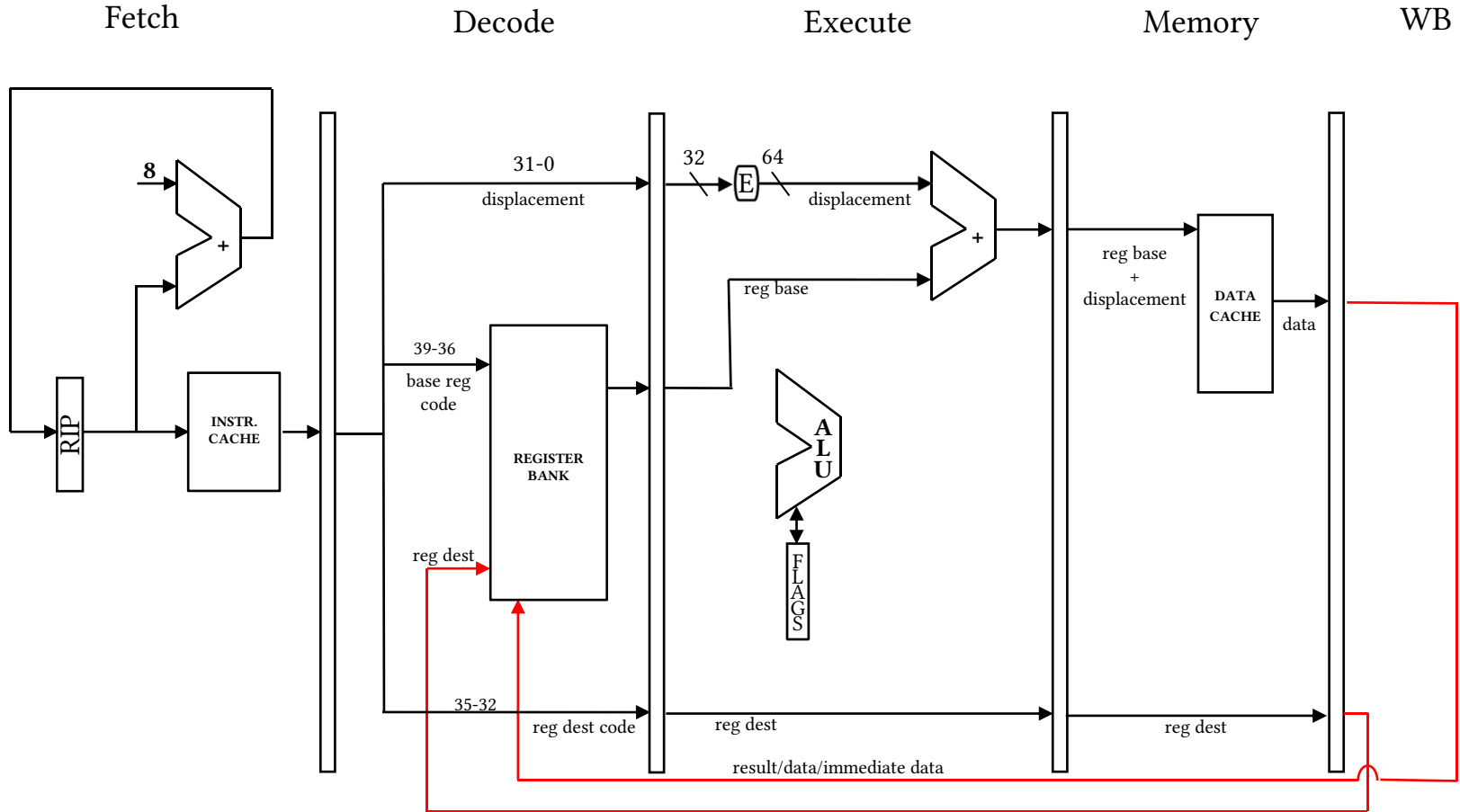
Istruzione Load: fase di execute



Istruzione Load: fase di memory

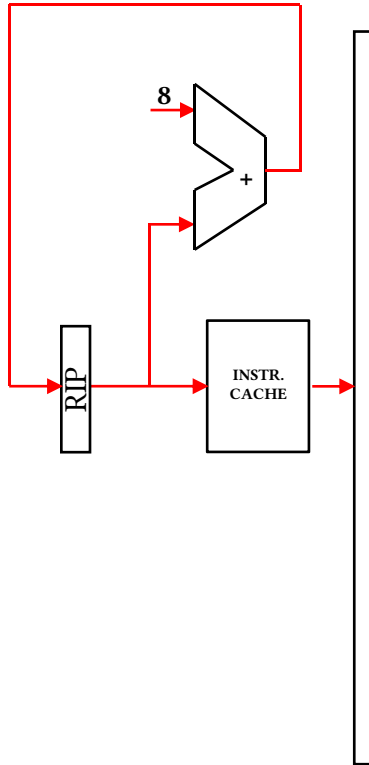


Istruzione Load: fase di write back

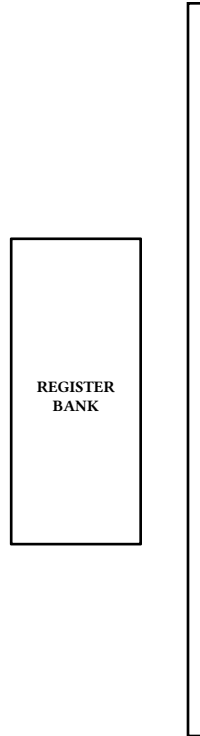


Istruzione Store: fase di fetch

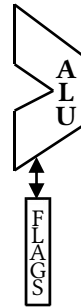
Fetch



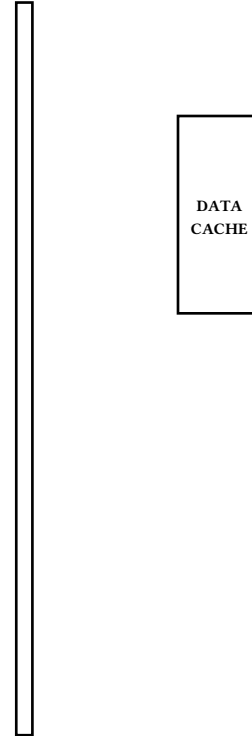
Decode



Execute



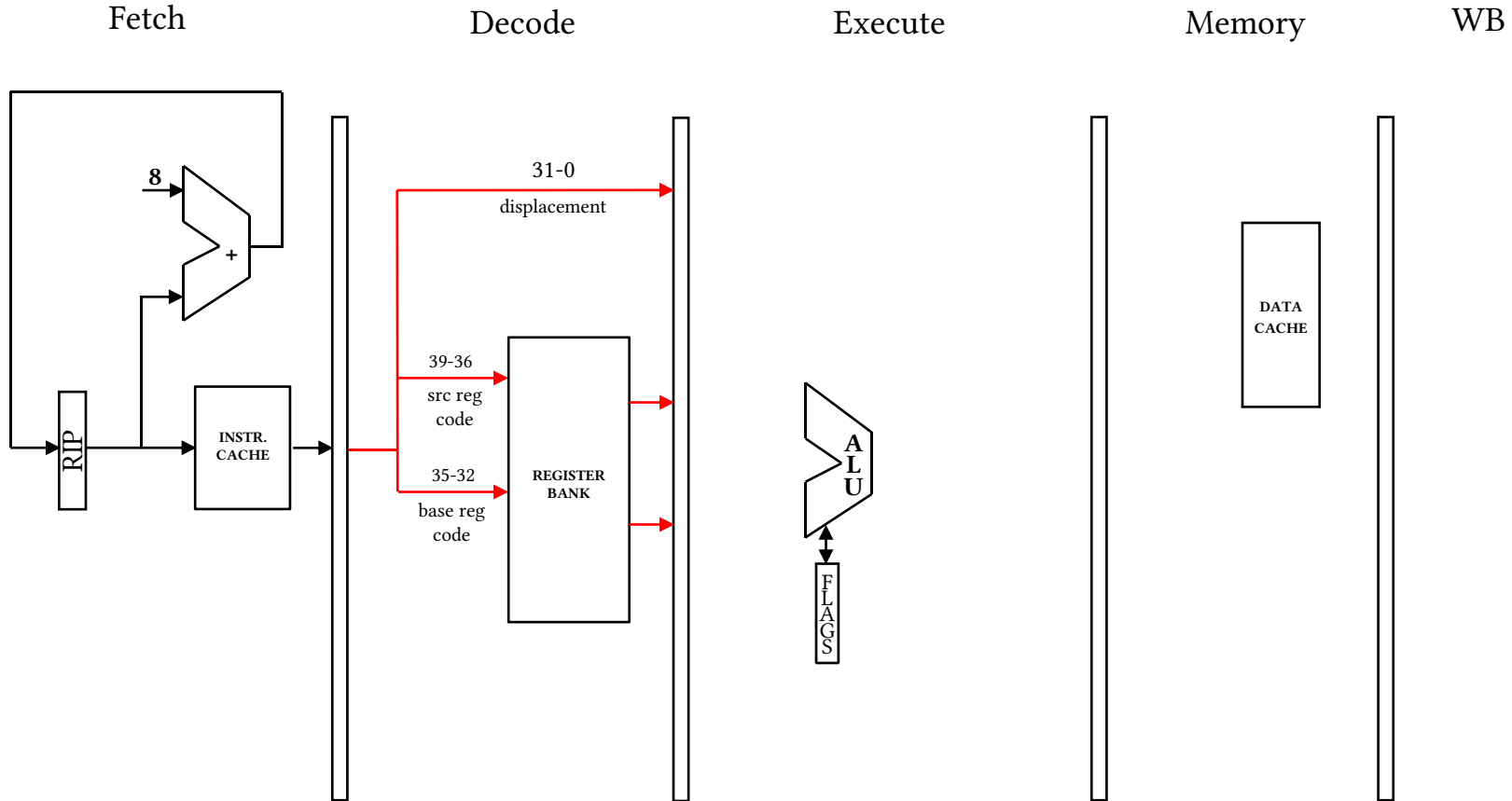
Memory



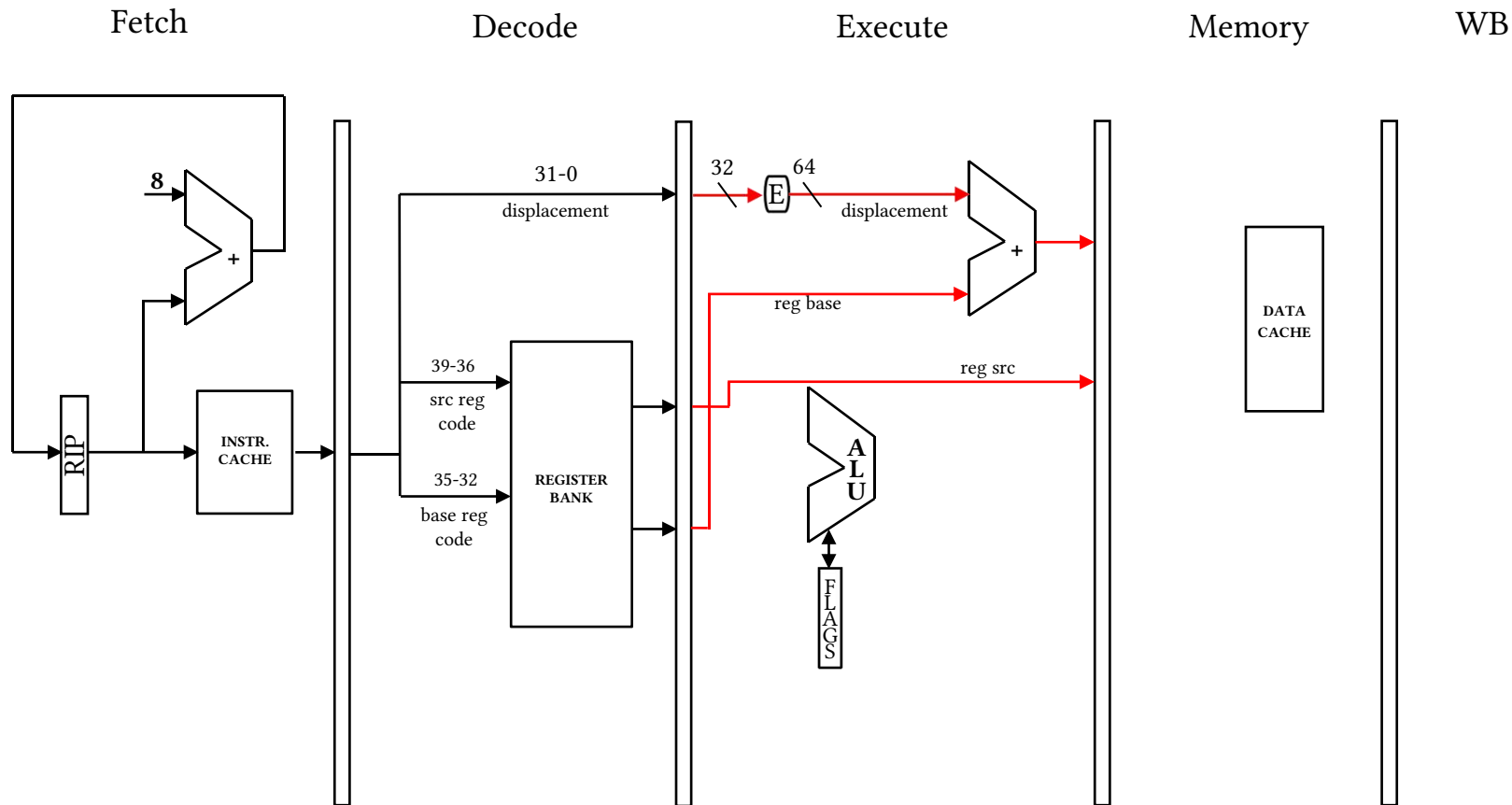
WB



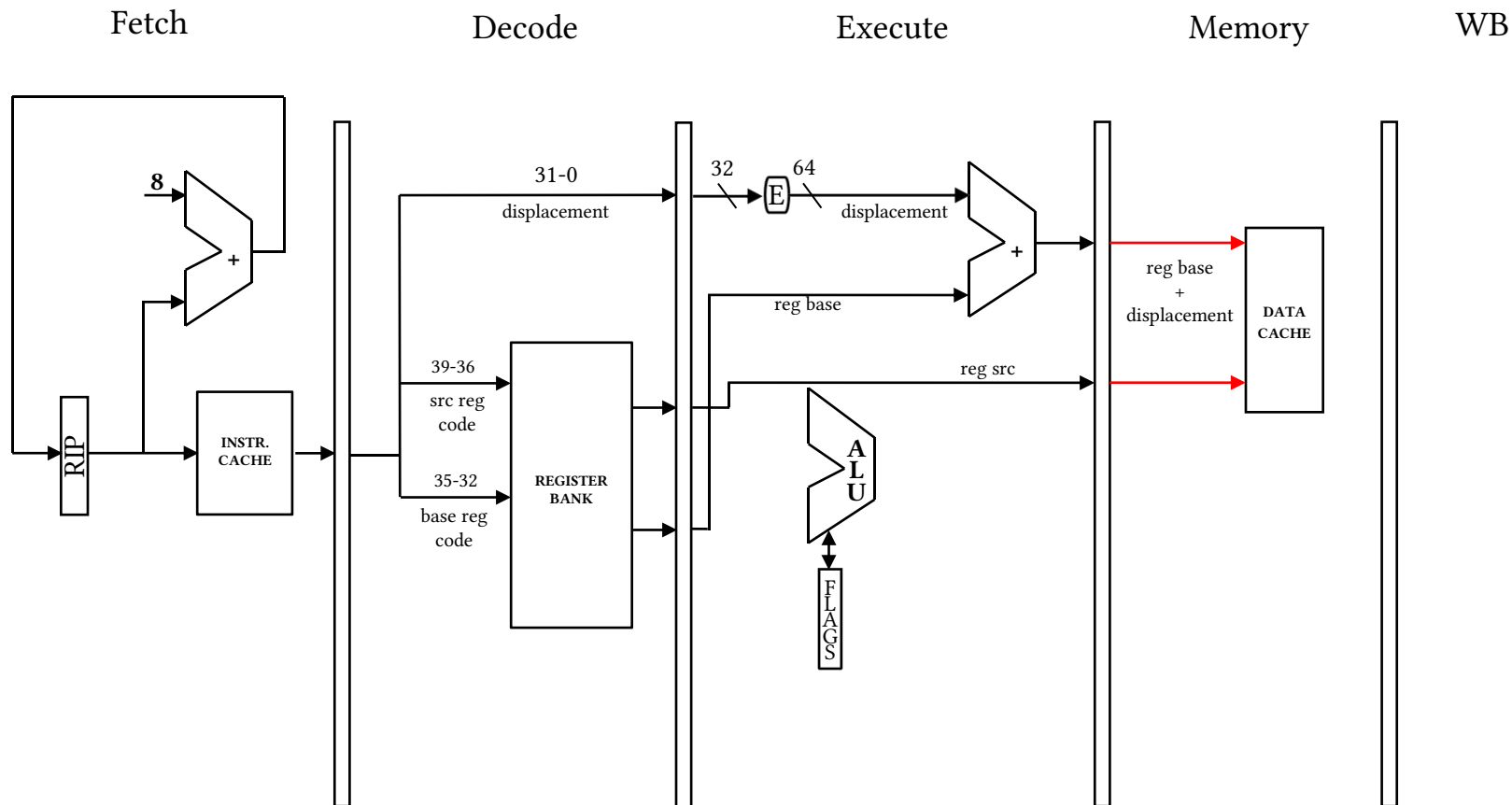
Istruzione Store: fase di decode



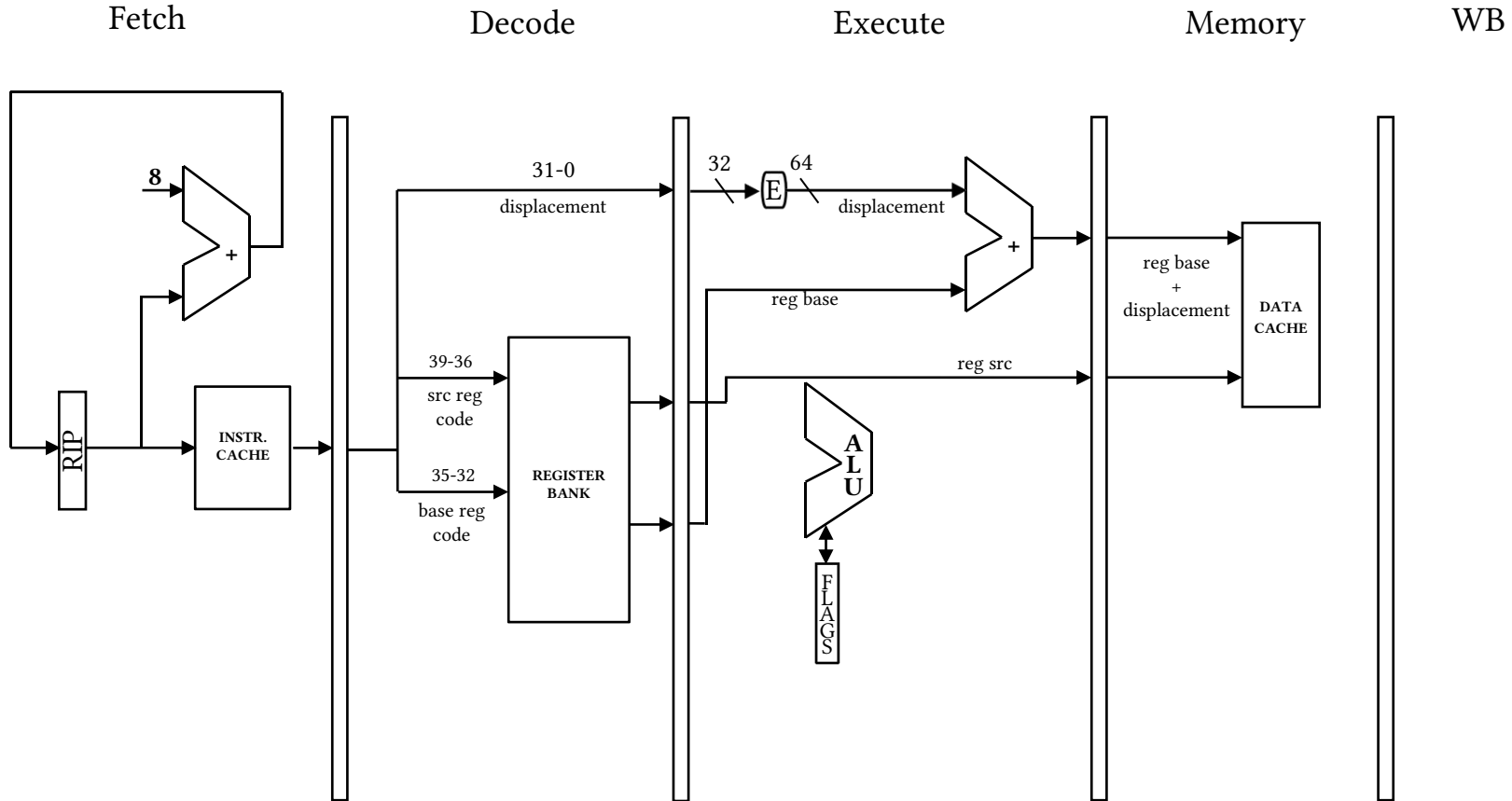
Istruzione Store fase di execute



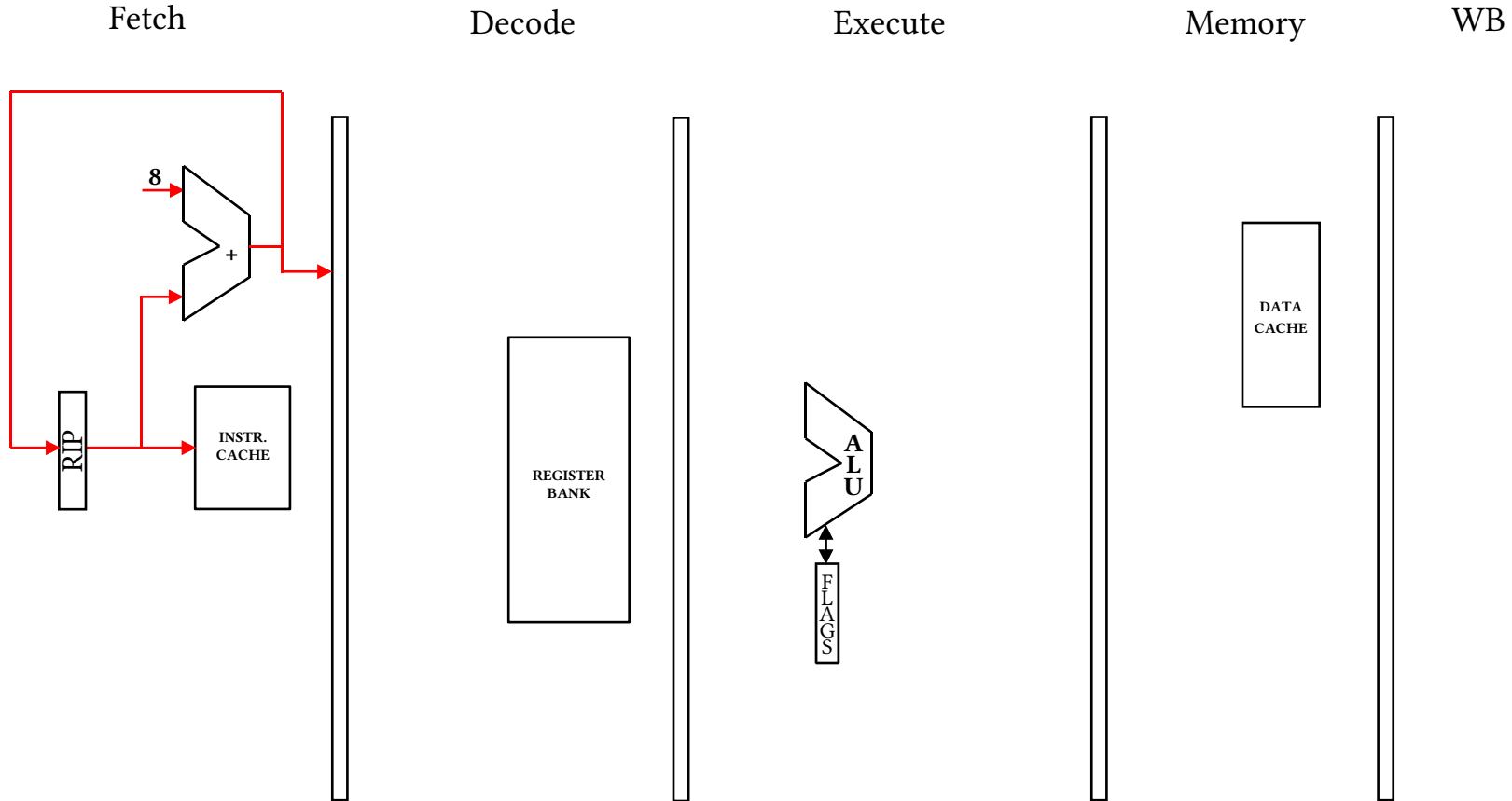
Istruzione Store fase di memory



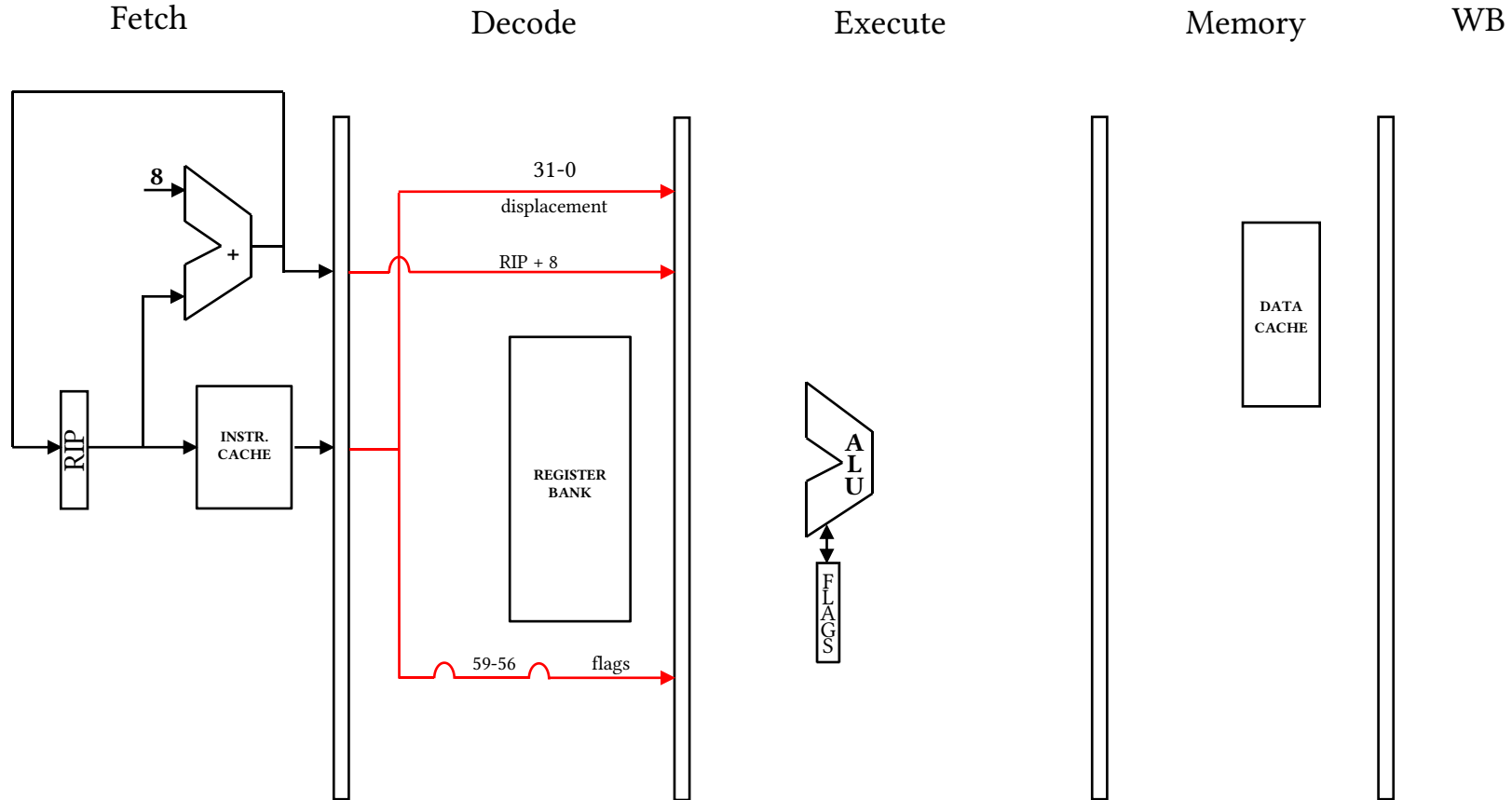
Istruzione Store fase di write back



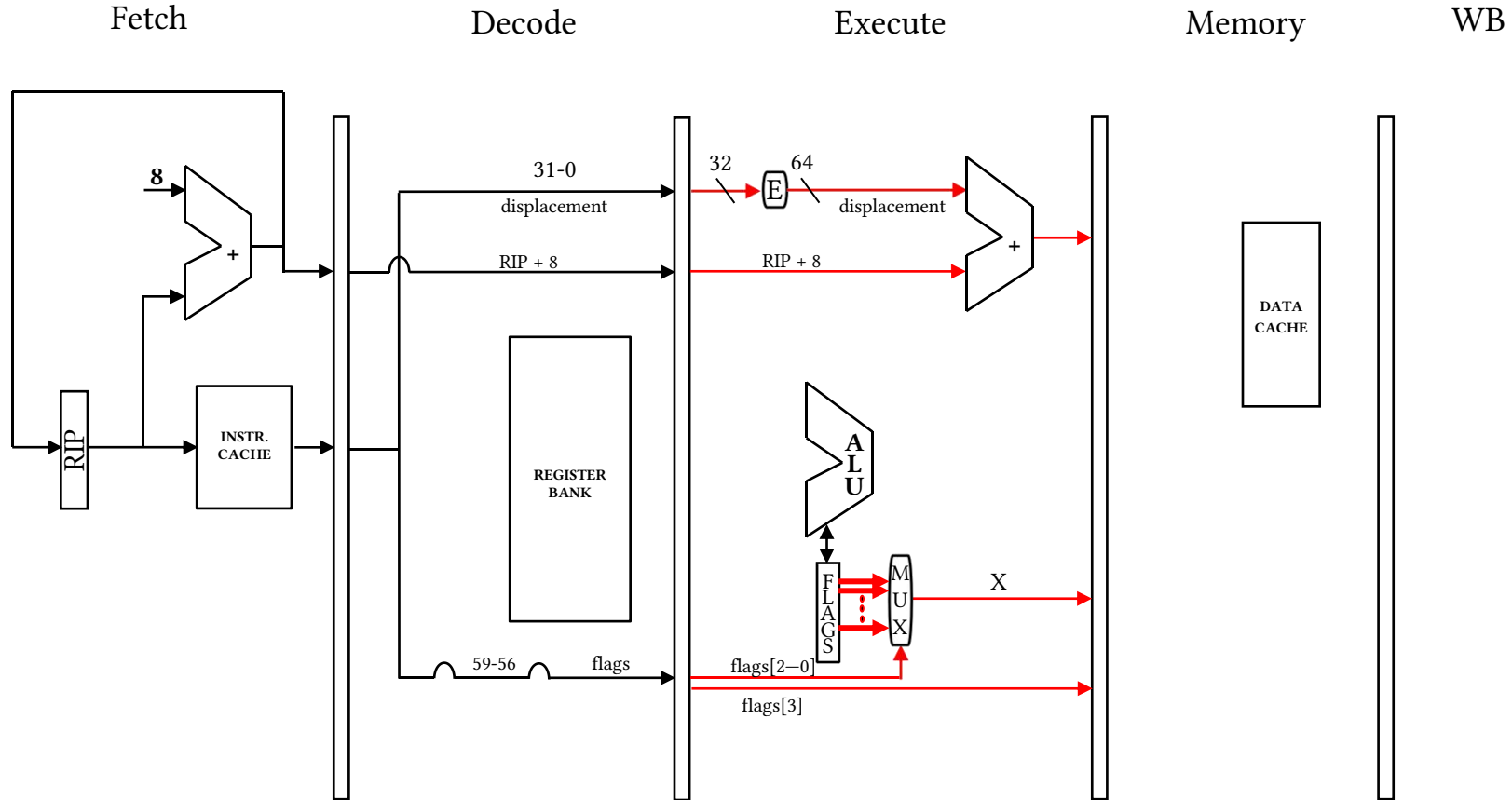
Istruzioni di salto condizionale: fase di fetch



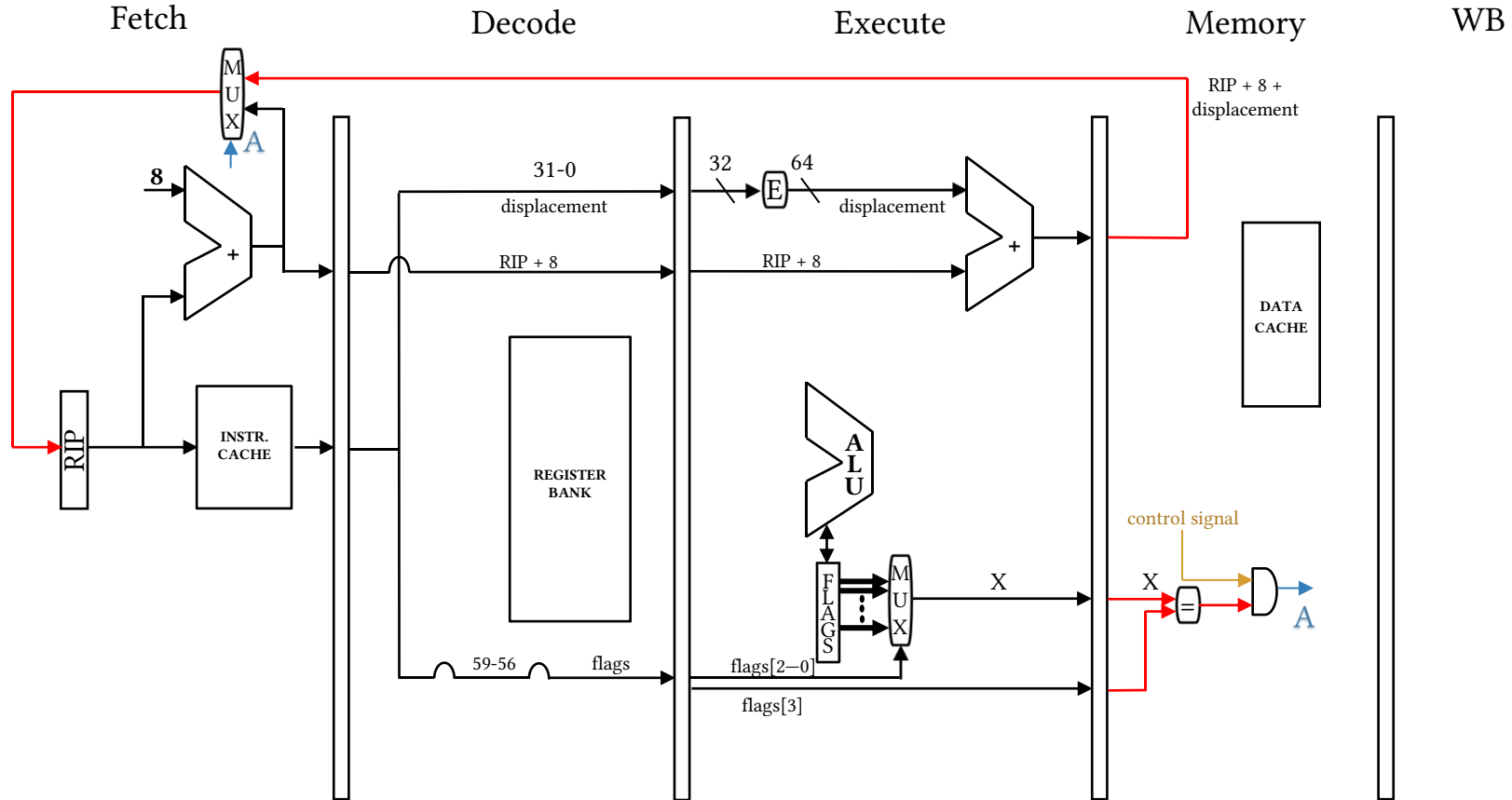
Istruzioni di salto condizionale: fase di decode



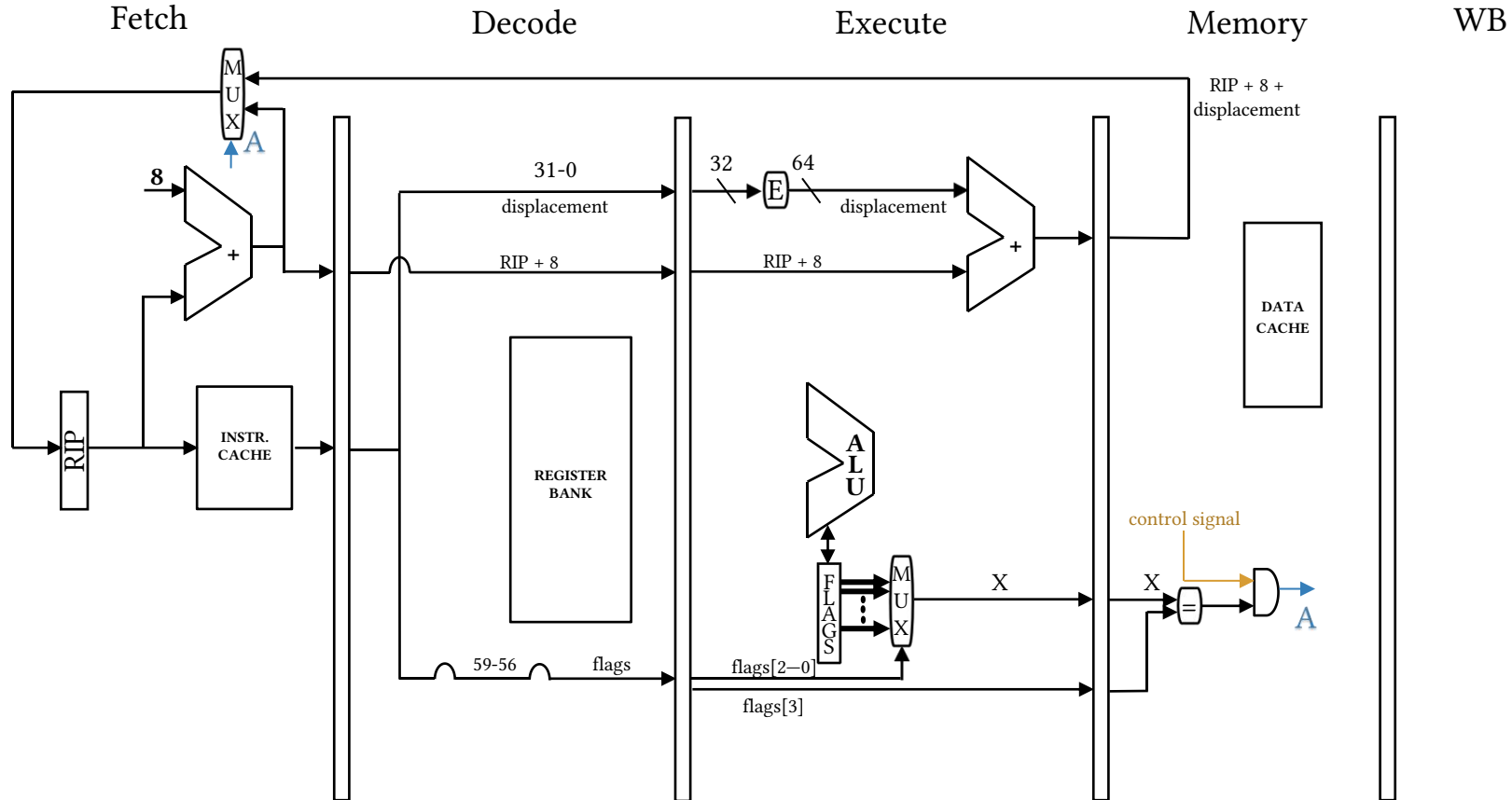
Istruzioni di salto condizionale: fase di execute



Istruzioni di salto condizionale: fase di memory

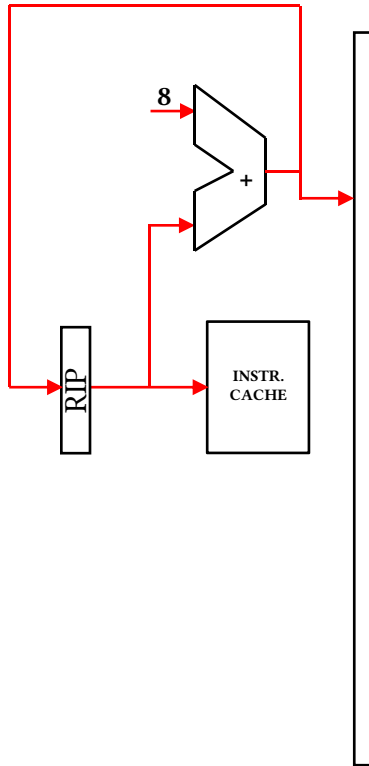


Istruzioni di salto condizionale: fase di write back

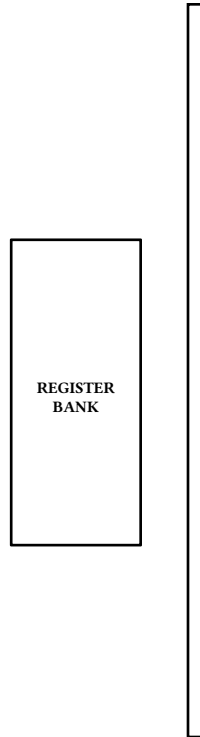


Istruzione nop: fase di fetch

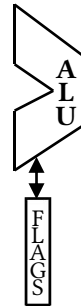
Fetch



Decode



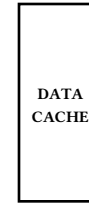
Execute



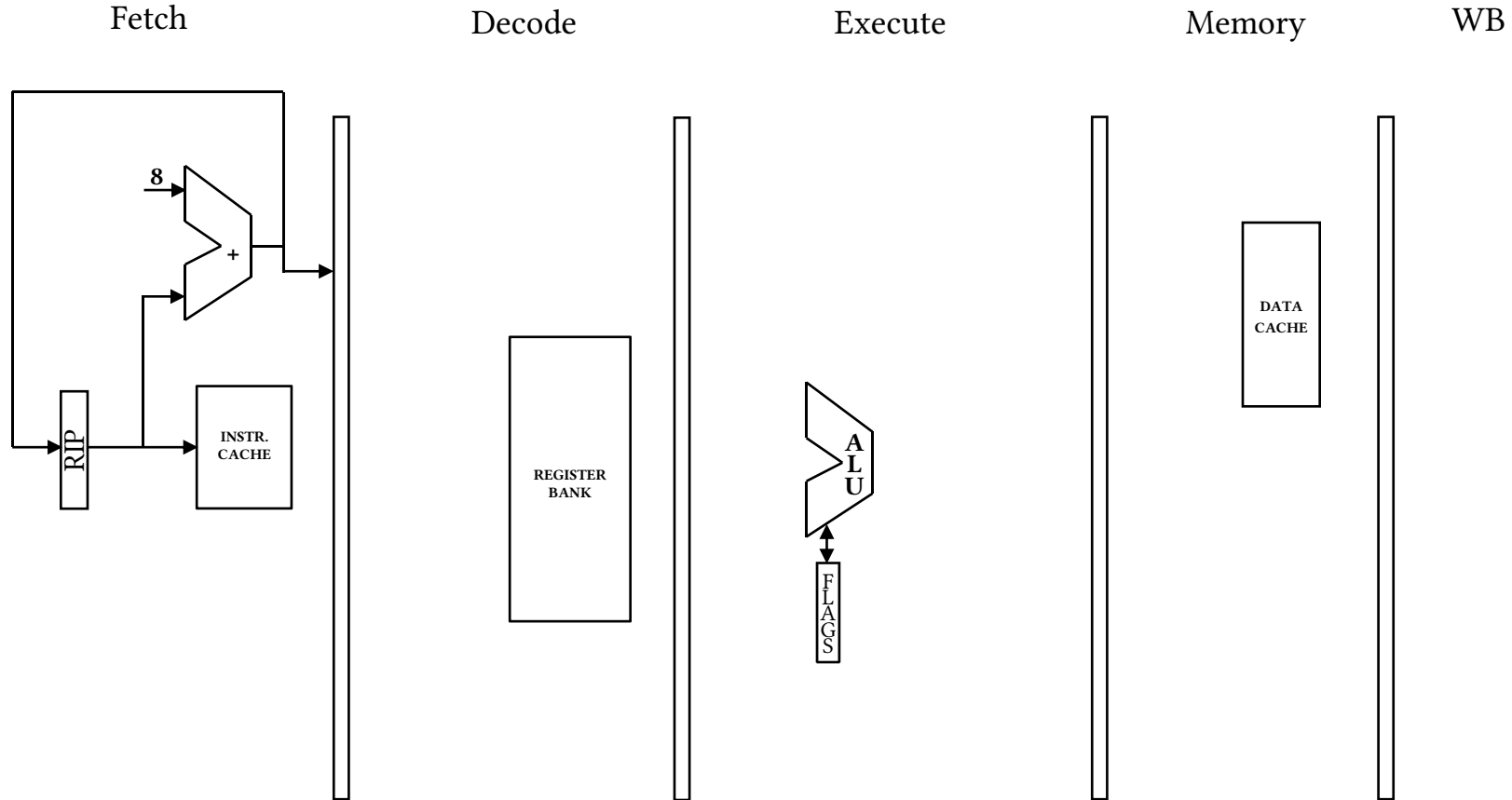
Memory



WB



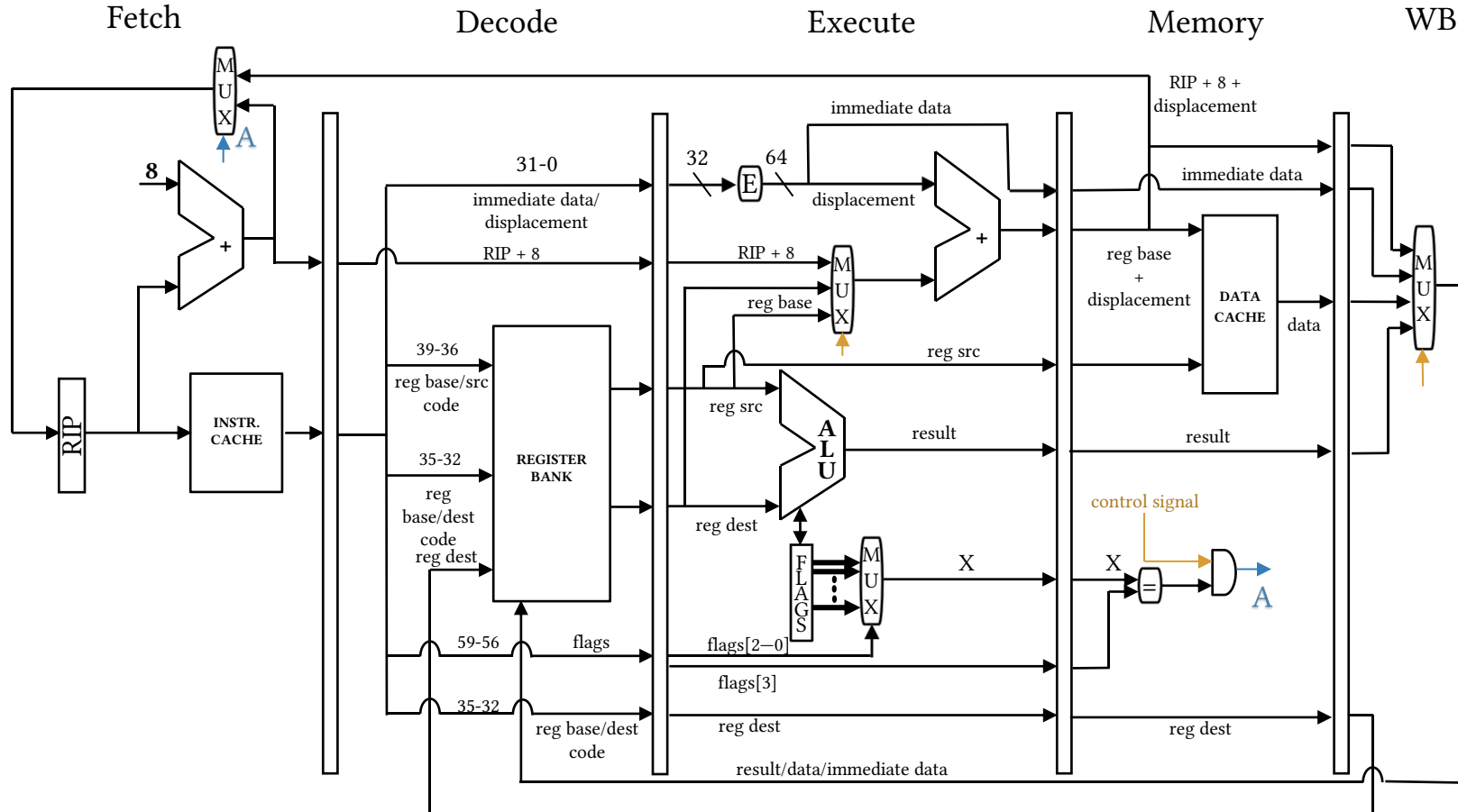
Istruzione nop: fasi successive



Datapath completo

- L'unità di calcolo finale può essere realizzata unendo insieme i datapath di tutte le singole classi di istruzioni
- Problema: ci sono alcuni percorsi in conflitto.
 - Esempio: la fase di write back delle istruzioni Load e L/A
 - Nel primo caso, occorre recuperare il valore propagato dalla cache dati
 - Nel secondo caso, occorre recuperare il valore propagato dalla ALU
- Soluzione: utilizzo di multiplexer per selezionare il valore corretto
- Tutti gli altri valori, comunque calcolati dagli stadi della pipeline, vengono ignorati

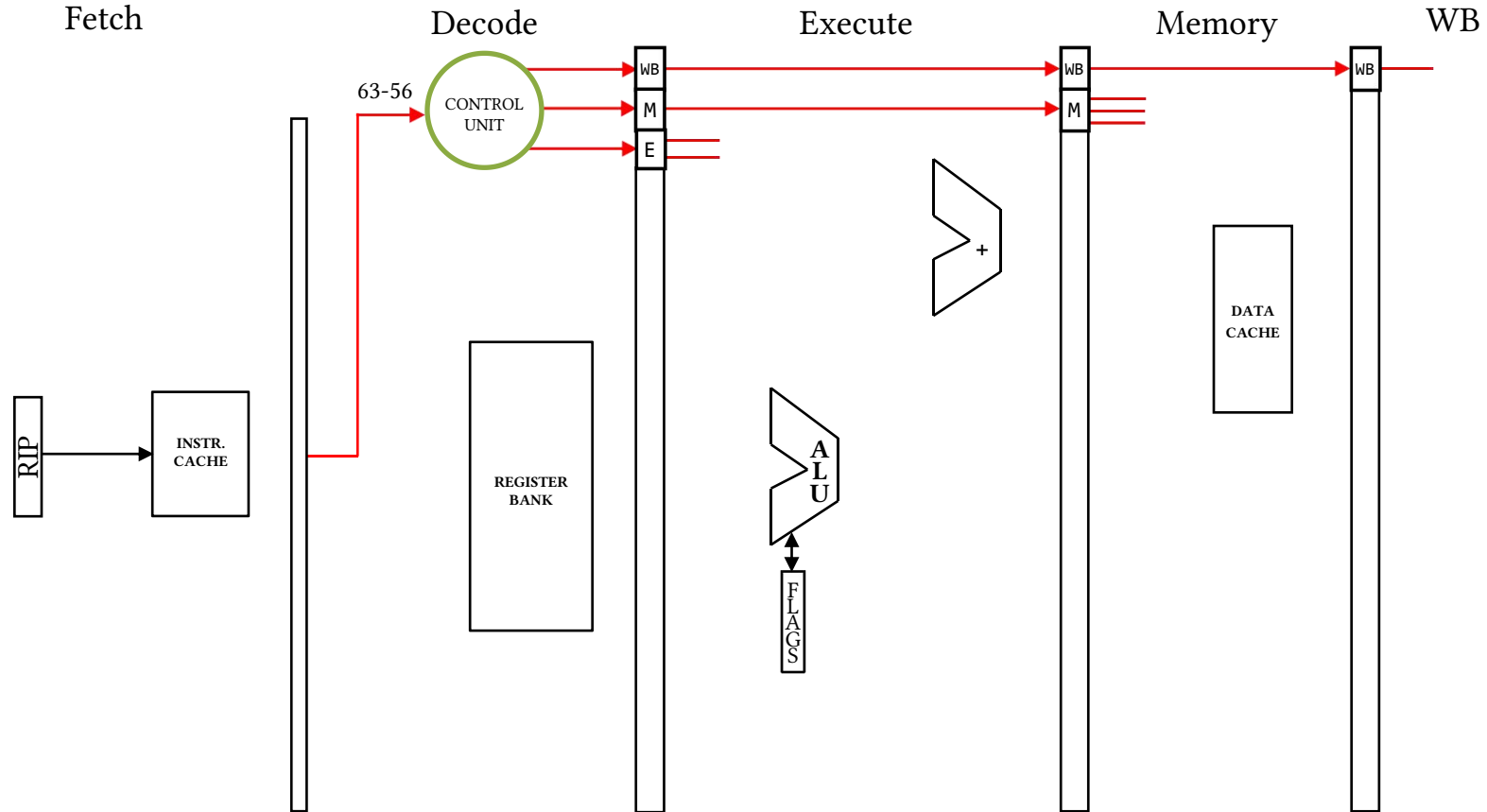
Datapath completo



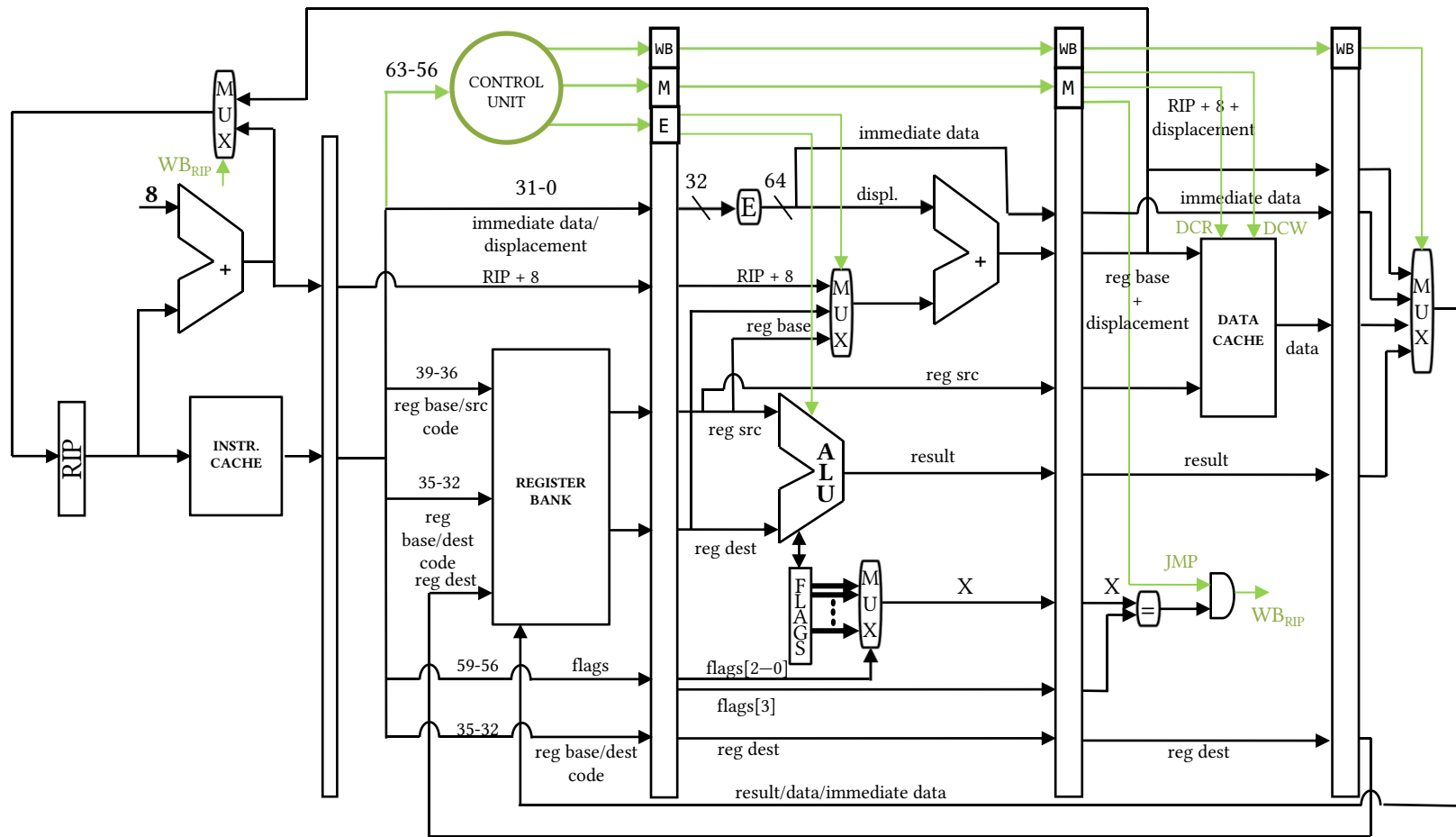
Unità di controllo

- I dati viaggiano attraverso gli stadi della pipeline
- Tutti i dati appartenenti ad un'istruzione devono essere mantenuti all'interno dello stesso stadio
- Le informazioni si trasferiscono solo tramite i registri della pipeline
- Le informazioni di controllo di una istruzione devono “viaggiare” con gli operandi / i dati dell'istruzione stessa
- Non sono necessari segnali di controllo per la scrittura dei registri di pipeline (si usa il clock), come per l'incremento di RIP

Posizionamento dell'unità di controllo



Architettura finale



Criticità nell'esecuzione delle istruzioni

- La propagazione dei valori pone delle problematiche nell'esecuzione delle istruzioni in un'architettura a pipeline
- Il risultato di un'istruzione potrebbe influenzare le istruzioni successive, ma tale risultato potrebbe non essere disponibile quando necessario

- Esempio:

```
addq %rax, %rbx  
movq %rbx, 0xabcd(%rdx)
```

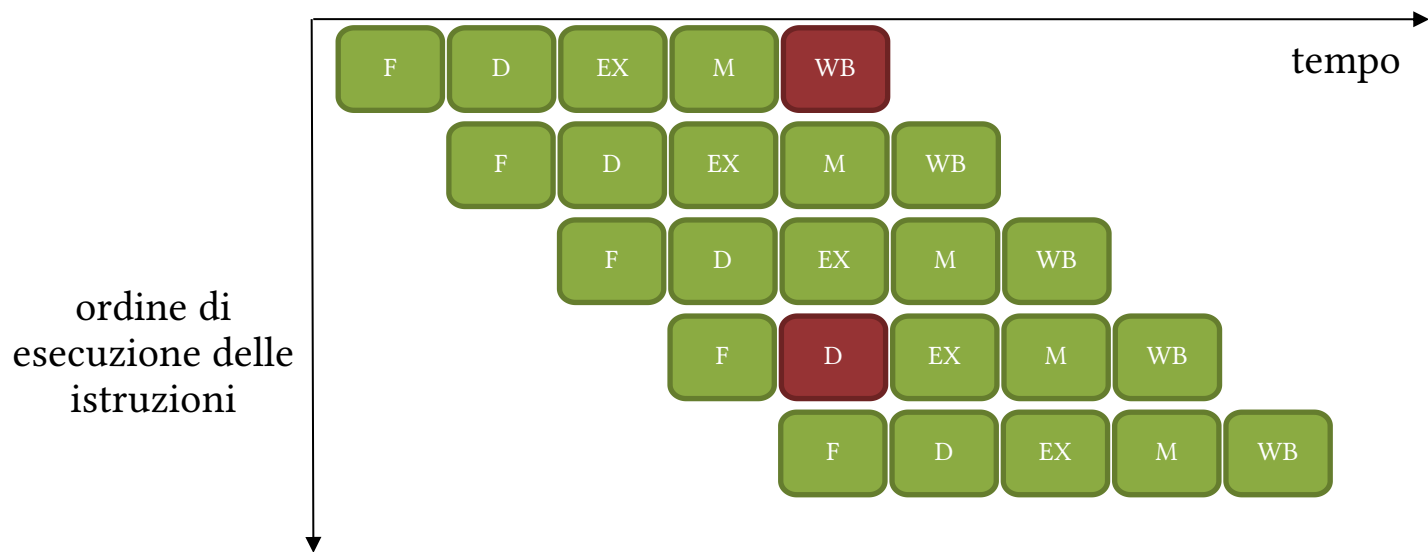
- Le *criticità* (o *conflitti* o *alee*) sorgono nelle architetture con pipelining quando non è possibile eseguire un'istruzione nel ciclo immediatamente successivo senza cambiare la semantica del programma eseguito in una struttura multiciclo

Le criticità

- Criticità **strutturale**
 - Tentativo di usare la stessa risorsa hardware da parte di diverse istruzioni in modi diversi nello stesso ciclo di clock
 - L'uso dell'architettura Harvard per la cache di primo livello elimina automaticamente la criticità strutturale per l'accesso a memoria dati e istruzioni.
- Criticità **sui dati**
 - Tentativo di usare un risultato prima che sia disponibile
- Criticità **sul controllo**
 - Nel caso di salti condizionali, decidere quale prossima istruzione sia da caricare dalla memoria prima che la condizione sia valutata

Criticità strutturali

- Il banco dei registri è soggetto a criticità strutturale
- Utilizzo contemporaneo con semantica differente



- Soluzione: scrittura sul fronte di salita, lettura sul fronte di discesa

Criticità sui dati

- Un'istruzione dipende dal risultato di un'istruzione precedente che è ancora nella pipeline

- Criticità **define-use**:

```
addq %rax, %rbx
```

```
subq %rax, %rbx
```

- Uno degli operandi della seconda istruzione dipende dal risultato della prima istruzione, che non è ancora stata *ritirata* dalla pipeline

- Criticità **load-use**:

```
movq 0xabcd(%rax), %rbx
```

```
subq %rbx, %rcx
```

- Uno degli operandi della seconda istruzione dipende dal valore letto dalla memoria, che non è ancora stata acceduta

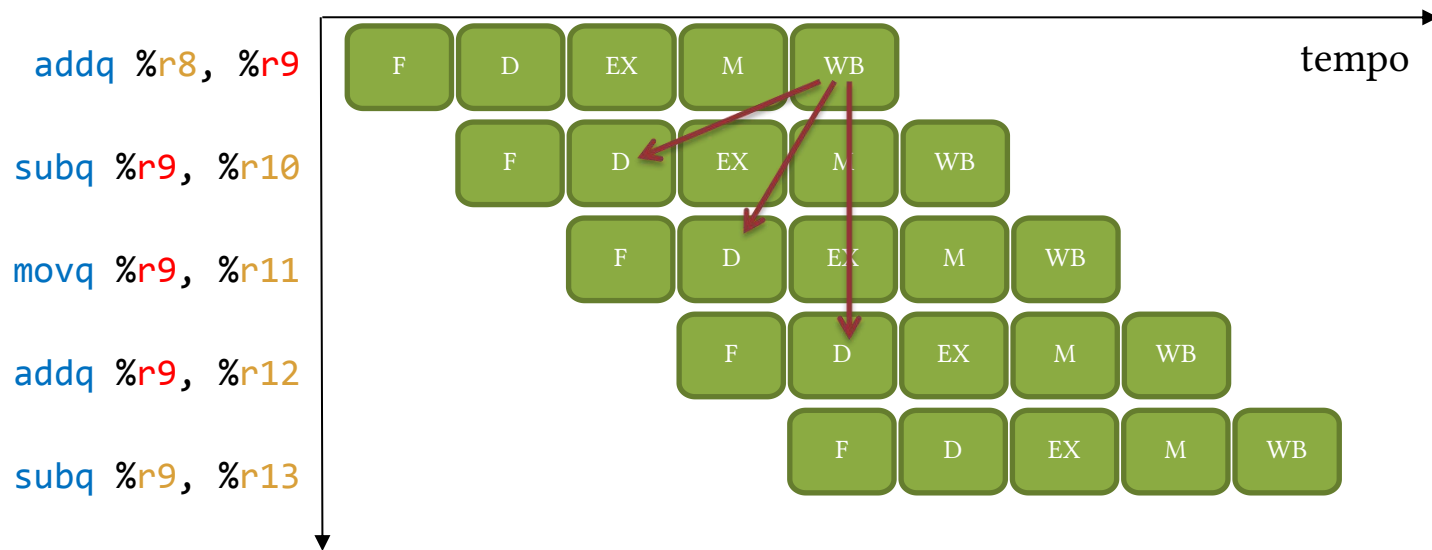
Frammento di programma con criticità define-use

```
addq %r8, %r9  
subq %r9, %r10  
movq %r9, %r11  
addq %r9, %r12  
subq %r9, %r13
```

- La criticità può coinvolgere più di una istruzione
- In generale, il numero di istruzioni che possono essere soggette a criticità dipende dal numero di stadi della pipeline

Frammento di programma con criticità define-use

- Lo schema temporale dell'esecuzione delle istruzioni permette di identificare le criticità
- La criticità define-use nella quarta istruzione è logicamente collegata alla criticità strutturale

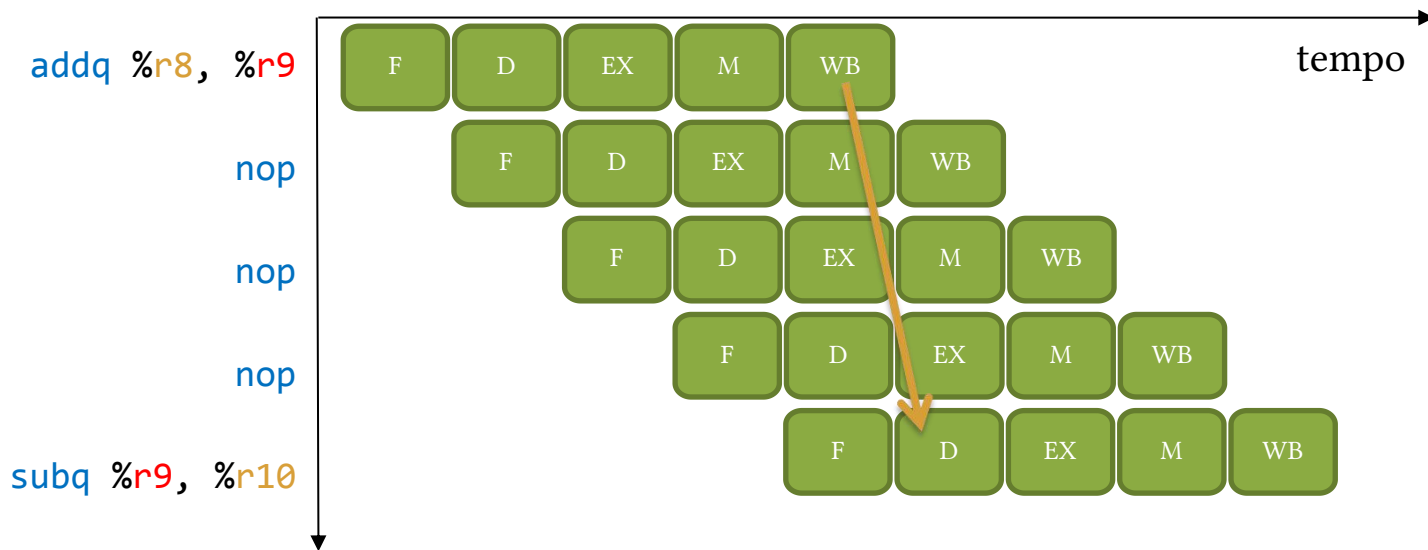


Soluzioni per le criticità sui dati

- Soluzioni di tipo software
 - inserimento di istruzioni nop
 - peggiora il throughput
 - riordino delle istruzioni
 - *spostare* istruzioni “innocue” in modo che esse eliminino la criticità
- Soluzioni di tipo hardware
 - inserimento di bolle (*bubble*) o stalli nella pipeline
 - si inseriscono dei tempi morti
 - peggiora il throughput
 - propagazione o scavalcamiento (*forwarding* o *bypassing*)
 - si propagano i dati in avanti appena sono disponibili verso le unità che li richiedono

Soluzione software: inserimento di nop

- Criticità define-use: l'assemblatore inserisce tra le istruzioni add e sub tre istruzioni nop, facendo così scomparire il conflitto
 - L'istruzione nop è l'equivalente software dello stallo



- Nel caso di assenza di criticità strutturale sul banco dei registri sono sufficienti due sole nop

Soluzione software: riordino delle istruzioni

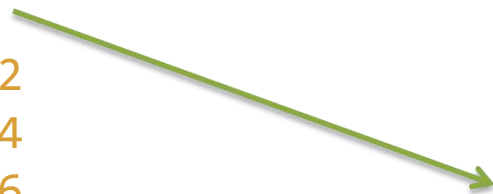
- L'assemblatore o il programmatore riordina le istruzioni in modo da impedire che istruzioni correlate siano troppo vicine
- L'assemblatore o il programmatore cerca di inserire tra le istruzioni correlate (che presentano dei conflitti) delle istruzioni *indipendenti* dal risultato delle istruzioni precedenti
 - Quando l'assemblatore non riesce a trovare istruzioni indipendenti deve inserire istruzioni nop

Frammento con criticità

```
addq %r8, %r9
subq %r9, %r10
subq %r11, %r12
addq %r13, %r14
subq %r15, %r16
```

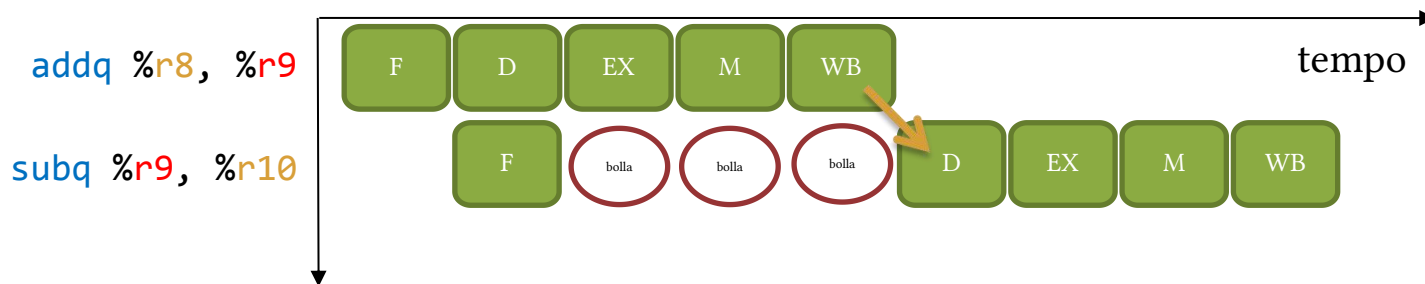
Frammento senza criticità

```
addq %r8, %r9
subq %r11, %r12
addq %r13, %r14
subq %r15, %r16
subq %r9, %r10
```



Soluzione hardware: inserimento di bolle

- Si inseriscono delle bolle nella pipeline, ovvero si blocca il flusso di istruzioni nella pipeline (*stallo*) finché il conflitto non è risolto
- Criticità define-use: occorre inserire tre bolle per fermare l'istruzione sub affinché possano essere letti i dati corretti
 - due bolle se vi è assenza di conflitti strutturali sul banco dei registri

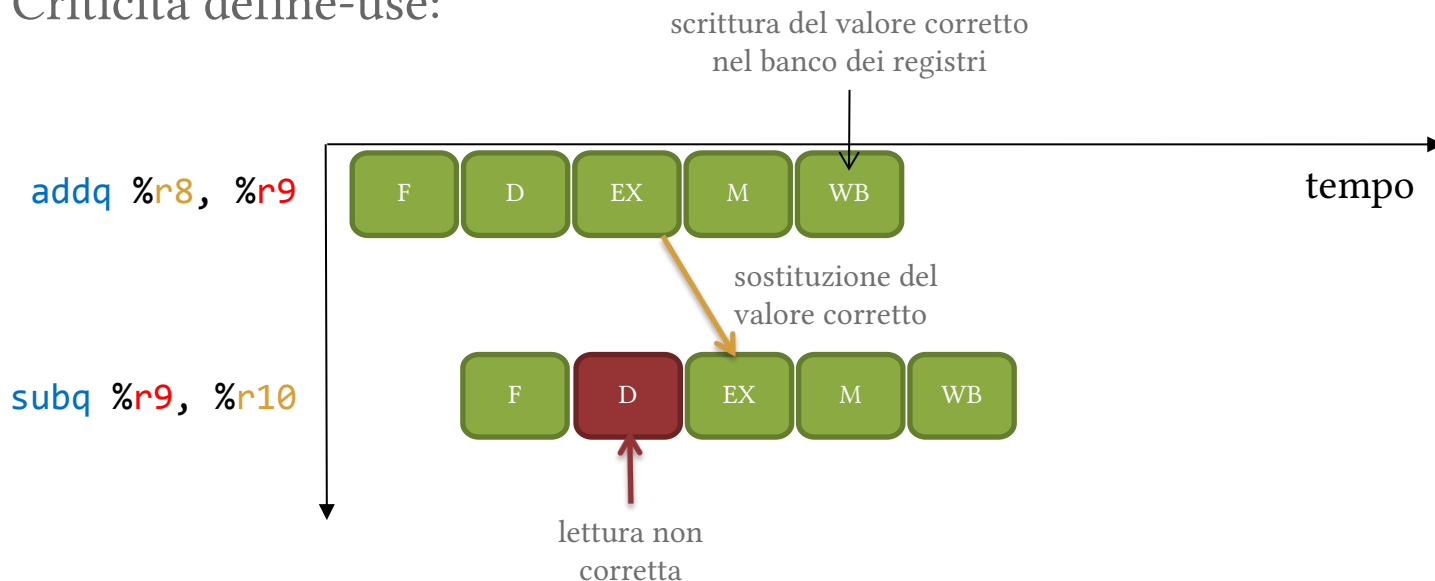


Soluzione hardware: inserimento di bolle

- Le bolle si generano:
 - bloccando il fetch delle istruzioni
 - facendo propagare il codice operativo della nop invece che quella dell'istruzione nello stato di DECODE
- Introduzione dell'*hazard detection unit*
 - Circuito combinatorio
 - La presenza dei conflitti viene rilevata comparando l'identificativo dei registri destinazione e sorgente e verificando che l'istruzione in uno stadio più avanti abbia una fase di write back

Soluzione hardware: propagazione in avanti

- Quando l'ALU genera il risultato, questo viene messo a disposizione dell'istruzione che segue sostituendo l'operando letto dal banco dei registri in fase di decode
- Criticità define-use:

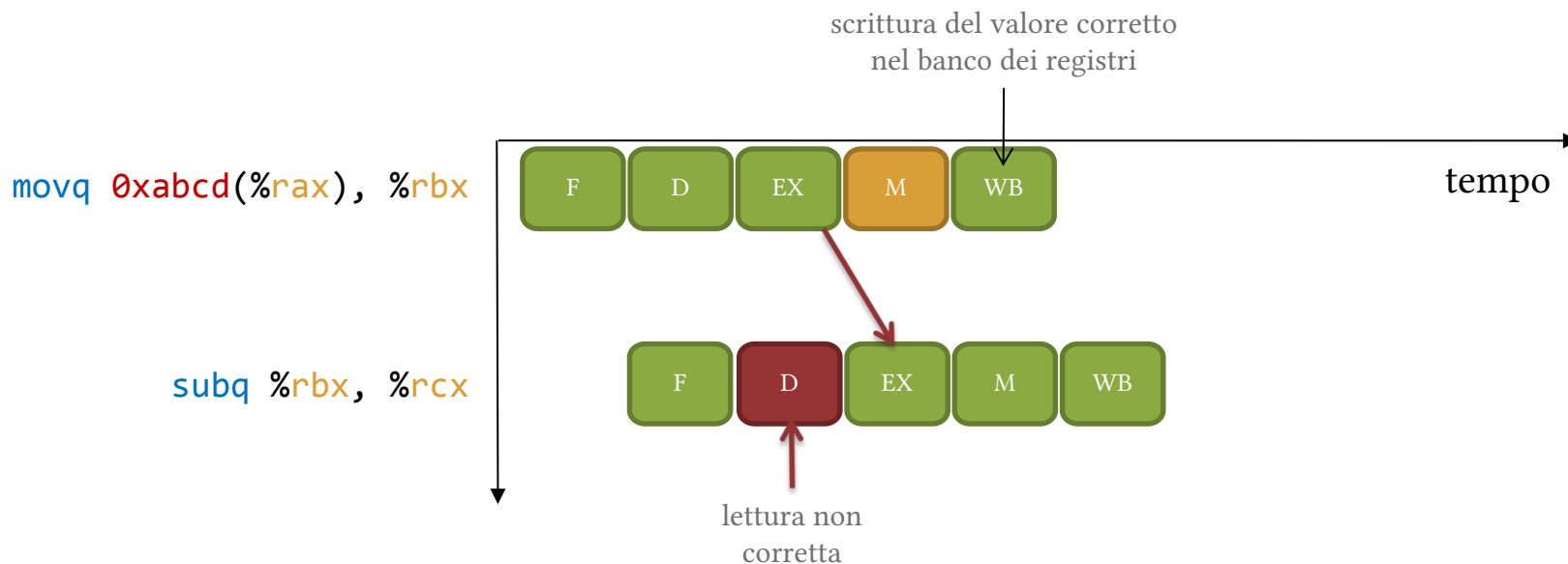


Criticità load-use

- Il dato caricato dall'istruzione di load non è ancora disponibile quando viene richiesto da un'istruzione successiva
- Le soluzioni software sono identiche al caso della criticità define-use
- L'inserimento di bolle tramite hazard unit è identica: occorre attendere il raggiungimento della fase di write back dell'istruzione precedente per procedere al decode della successiva
- La soluzione basata su propagazione in avanti è differente
 - Il dato corretto è disponibile al termine della fase di memory

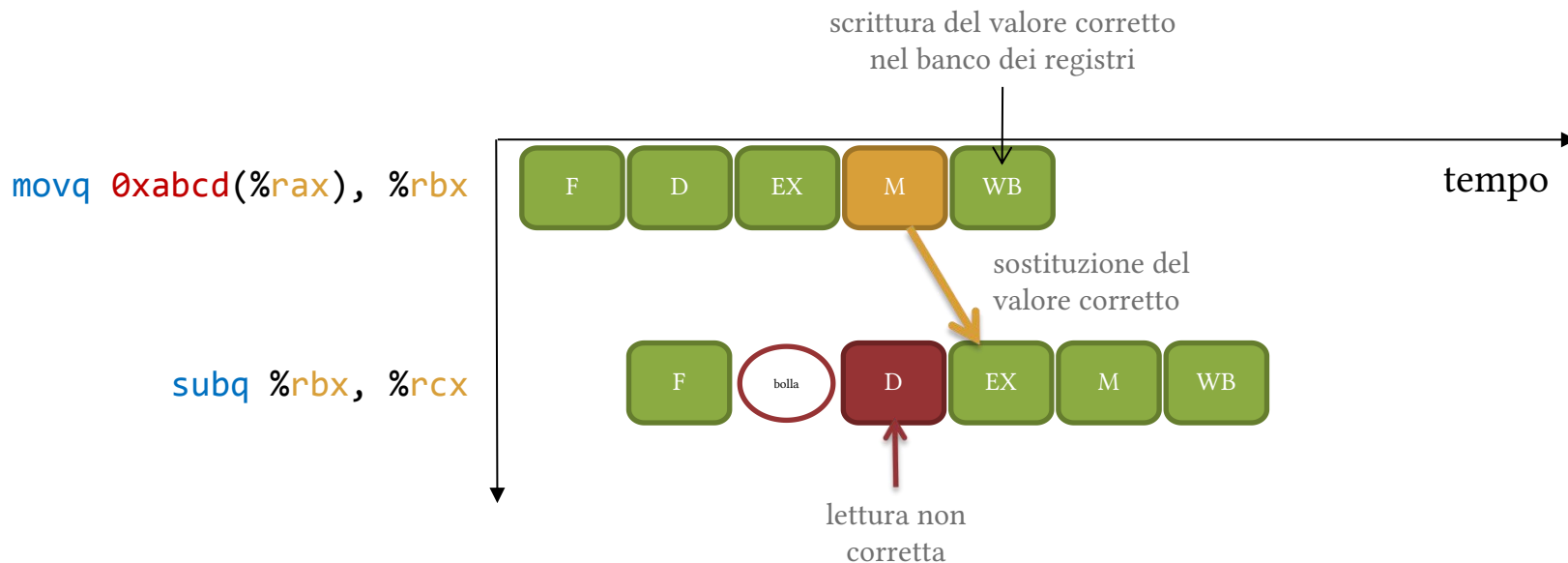
Criticità load-use: propagazione incorretta

- Se si utilizza la stessa strategia usata per la criticità define-use, il risultato non è corretto



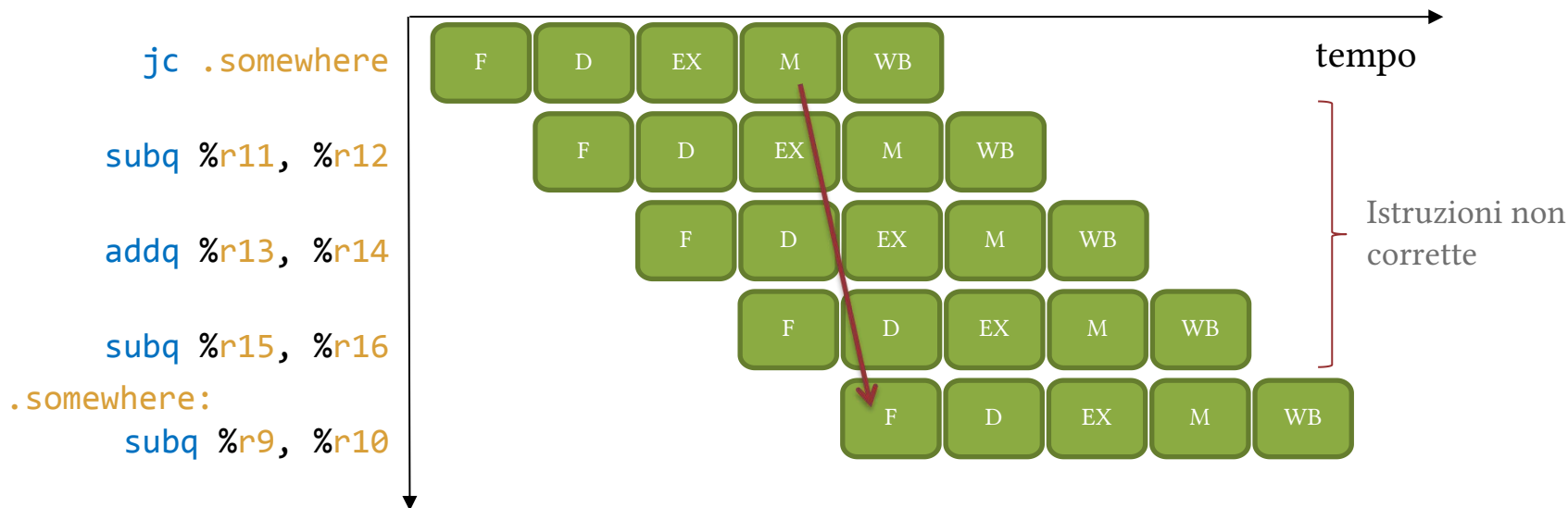
Criticità load-use: propagazione corretta

- È necessario inserire una bolla per consentire la corretta propagazione



Criticità sul controllo

- La pipeline viene alimentata effettuando il fetch dell'istruzione successiva ($RIP \leftarrow RIP + 8$)
- Se viene effettuato un salto condizionale, il valore di RIP viene aggiornato durante la fase di memory



Soluzioni per le criticità sul controllo

- Approccio *pessimistico*: non si eseguono istruzioni fino al completamento della scelta
 - Vantaggio: implementazione più semplice
 - Svantaggio: spreco di cicli di clock (CPI aumenta inesorabilmente)
- Approccio *ottimistico*: si prospetta che non venga effettuato il salto
 - Vantaggio: se la condizione di salto non è verificata, l'esecuzione ha le migliori prestazioni
 - Se il salto viene effettuato, le prestazioni sono analoghe all'approccio pessimistico
 - Svantaggio: necessità di annullare gli effetti delle istruzioni eseguite, nel caso in cui il salto sia effettuato
 - Maggiore complessità dell'unità di controllo e dell'unità di calcolo
 - Esempio: registro FLAGS

Soluzione hardware ottimistica

- Caso peggiore: il salto viene effettuato
1. Annullare l'esecuzione delle istruzioni che sono erroneamente entrate nella pipeline (*flush* della pipeline)
 - Reset dei registri di interfaccia, simulando l'esecuzione di una nop
 2. Ripristino del contenuto del registro FLAGS
 - È necessario mantenere una copia del contenuto del registro
 - È possibile utilizzare un registro tampone aggiuntivo
 - In tale registro viene scritta una copia del valore di FLAGS prima che esso venga aggiornato per l'esecuzione di un'istruzione
 - In caso di flush della pipeline, il valore precedente viene ripristinato

Ottimizzazione delle prestazioni: predizione del salto

- Quando si carica un'istruzione di salto condizionale, si cerca di *prevedere* se il salto sarà effettuato o meno
 - Se la previsione è corretta, si massimizzano le prestazioni
 - Se la previsione non è corretta, si effettua il flush della pipeline
- È possibile realizzare predizioni dinamiche o statiche
- Predizione *dinamica*:
 - Il valore previsto cambia durante l'esecuzione del programma
 - Necessita un'implementazione hardware (Branch Prediction Unit)
 - Tipicamente basato sullo *storico di salto*
 - Semplice apporccio: predici che il salto sarà effettuato se è stato effettuato in precedenza
- Predizione *statica*:
 - Salto considerato effettuato solo verso parti precedenti di codice (cicli)
 - Guidata dal compilatore
 - Guidata dal programmatore (es: prefissi 0x2e, 0x3e del Pentium 4)

Tabella di predizione del salto

- Piccola memoria indicizzata dai bit meno significativi dell'indirizzo dell'istruzione di salto condizionale
- Ogni istruzione è associata ad una previsione
 - *Effettuare* o *non effettuare* il salto
- Se la previsione è “*effettua*” ed è corretta:
 - Penalità di un solo ciclo
- Se la previsione è “*non effettuare*” ed è corretta:
 - Nessuna penalità
- Se la previsione è *errata*:
 - Modificare la previsione
 - Effettuare il flush della pipeline
 - La penalità è la stessa che si avrebbe se non ci fosse alcuna predizione

Two-bit Saturating Counter

- Si utilizza una macchina a stati associata ad una singola istruzione di salto
- Per rendere più stabili le previsioni, si effettua il cambio di previsione dopo almeno due previsioni errate

